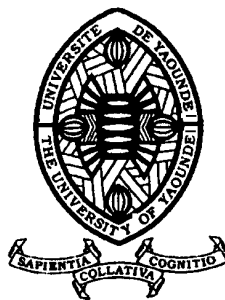


UNIVERSITÉ DE YAOUNDE I

ECOLE NATIONALE SUPERIEURE
POLYTECHNIQUE DE YAOUNDE

DEPARTEMENT DE GENIE
INFORMATIQUE

HUMANITÉS NUMÉRIQUES



UNIVERSITY OF YAOUNDE I

NATIONAL ADVANCED SCHOOL
OF ENGINEERING OF YAOUNDE

DEPARTMENT OF COMPUTER
ENGINEERING

DIGITAL HUMANITIES

CONSOMMATION ÉNERGÉTIQUE LIÉE À LA PILE RÉSEAU DU NOYAU LINUX : CAS DES APPLICATIONS WEB

MÉMOIRE DE FIN D'ÉTUDES

Présenté et soutenu par :

ATANGANA WAMSA IADINE BRENDA

En vue de l'obtention du :

Diplôme d'Ingénieur de Conception en Humanités Numériques

Option Cybersécurité et Investigation Numérique

Sous la supervision de :

Thomas DJOTIO NDIÉ, Maître de conférence, Université de Yaoundé 1

Thystere MVONDO DJOB, Maître de conférences, Université de Rennes

Césaire HONORÉ MOUNAH, Doctorant, Université de Rennes

Devant le jury composé de :

Président : **Thomas BOUETOU BOUETOU**, Professeur, Université de Yaoundé 1

Rapporteur : **Thomas DJOTIO NDIÉ**, Maître de Conférences, Université de Yaoundé 1

Examineur : **Guy Merlin NGOUNOU**, Chargé de cours, Université de Yaoundé 1

Invités : **Thystere MVONDO DJOB**, Maître de conférences, Université de Rennes

Césaire Honoré MOUNAH, Doctorant, Université de Rennes

Année académique 2025-2026

Février 2026



**CONSOMMATION ÉNERGÉTIQUE LIÉE À LA PILE
RÉSEAU DU NOYAU LINUX : CAS DES APPLICATIONS
WEB**

MÉMOIRE DE FIN D'ÉTUDES

Présenté et soutenu par :

ATANGANA WAMSA IADINE BRENDA

En vue de l'obtention du :

DIPLÔME D'INGÉNIEUR DE CONCEPTION EN HUMANITÉS NUMÉRIQUES

Option Cybersécurité et Investigation Numérique

Sous la supervision de :

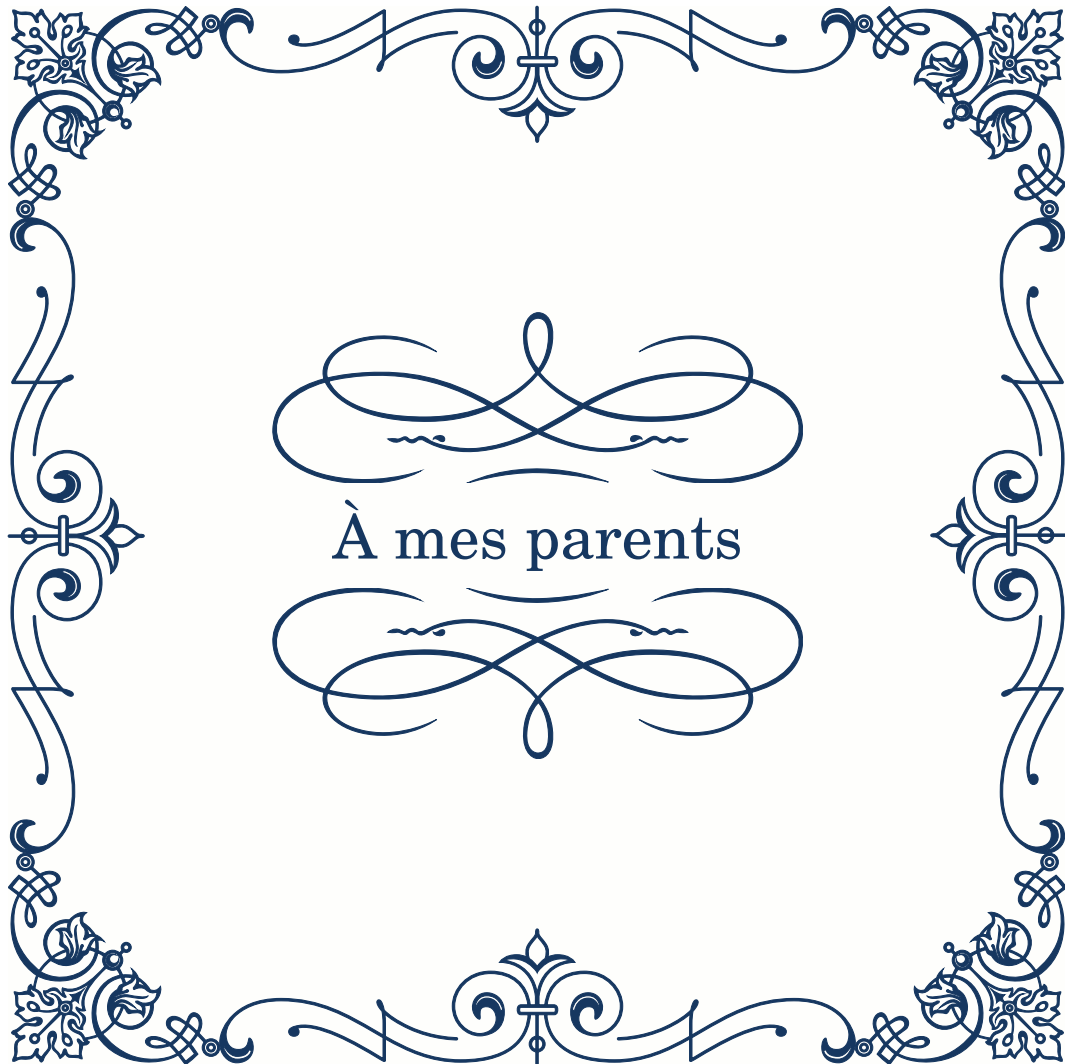
Thomas DJOTIO NDIÉ, Maître de conférences, Université de Yaoundé 1

Thystere MVONDO DJOB, Maître de conférences, Université de Rennes

Césaire HONORÉ MOUNAH, Doctorant, Université de Rennes

Année académique 2025-2026
Février 2026

DÉDICACE



REMERCIEMENTS

Je souhaite exprimer ma profonde gratitude et reconnaissance envers toutes les personnes qui, de près ou de loin, ont contribué à mon éducation , à ma formation et à la réalisation de ce travail, notamment :

- **Pr. Thomas BOUETOU BOUETOU**, pour l'honneur qu'il me fait en acceptant de présider ce jury.
- **Dr. Guy Merlin NGOUNOU**, pour l'honneur qu'il me fait en acceptant d'examiner scientifiquement ce travail.
- **Pr. Thomas DJOTIO NDIÉ** mon encadrant académique, pour sa disponibilité, ses observations, ses conseils et l'attention qu'il a portée à la rédaction de ce mémoire ;
- **Pr Thystère DJOB MVONDO** et **Mr Césaire Honoré Mounah**, mes encadrants professionnels, merci pour les précieux conseils, la confiance, l'assistance et le soutien permanent.
- Le laboratoire **IRISA** par le biais de son Directeur, **Pr Guillaume GRAVIER**, qui m'a fourni un cadre de travail agréable pour la réalisation de ce projet.
- Le chef de département du Génie Informatique **Pr Thomas BOUETOU BOUETOU** et le corps enseignant de l'ENSPY, pour la formation et le savoir qu'ils se sont dévoués à me transmettre.
- Mes parents, **Mr WAMSA Joseph** et **Mme WAMSA Madeleine**, pour leur amour, soutien, éducation, conseils et présence dans ma vie.
- Mes frères et sœur pour leurs encouragements.
- **Mme Victoire NGANFANG**, **Mr Caleb FONYUY-ASHERI**, **Mr Cyrille KENFACK** et toute l'équipe **WIDE** du laboratoire IRISA pour leur soutien tant sur le plan professionnel que social, qui a grandement facilité mon intégration sociale et culturelle au sein de la ville de Rennes.
- Mes camarades de la promotion HN-2025, amis et aînés académiques, notamment **YIAGNIGNI ABDEL** pour tous leurs conseils.

Je tiens enfin à remercier toutes les personnes que je n'ai pas pu citer et qui ont participé, d'une manière ou d'une autre à la réalisation de ce travail.

RÉSUMÉ

Dans un contexte marqué par la croissance rapide des applications web et les exigences de plus en plus accrues des utilisateurs, la consommation énergétique des infrastructures serveurs constitue un enjeu majeur, tant sur le plan économique qu'environnemental. Ce mémoire propose une approche fondée sur la configuration de l'affinité CPU, visant à contrôler le nombre de cœurs sur lesquels une application web s'exécute sous Linux, afin de réduire la consommation énergétique tout en préservant la qualité de service. Ce travail de recherche est réalisé en adoptant une démarche expérimentale et comparative, appliquée à deux serveurs d'applications web, Nginx et Node.js, dans des scénarios contrôlés et reproductibles. Les performances applicatives (latence et débit), la consommation énergétique et des métriques bas niveau ont été mesurées à l'aide d'outils de profilage système. Les résultats montrent que, pour Nginx, utilisé avec une charge majoritairement orientée entrées/sorties, l'affinité CPU a un impact limité sur les performances et l'énergie consommée. En revanche, pour une application Node.js à calcul intensif, une affectation contrôlée à un ensemble réduit de cœurs permet, dans certains cas, d'améliorer simultanément l'efficacité énergétique et les performances. À l'inverse, des migrations fréquentes entre cœurs se traduisent par une dégradation des métriques observées et une augmentation de la consommation énergétique. Ces résultats montrent que la méthode proposée est en mesure de réduire l'énergie consommée par les serveurs de 0,41% par rapport à l'utilisation de l'affinité CPU par défaut prédéfinie par le système d'exploitation Linux, tout en respectant la qualité de services de l'application web.

Mots clés : Consommation énergétique, Performance, Application web, Linux, Affinité CPU.

ABSTRACT

In a context marked by the rapid growth of web applications and increasingly demanding user requirements, the energy consumption of server infrastructures is a major challenge, both economically and environmentally. This thesis proposes an approach based on CPU affinity configuration, aimed at controlling the number of cores on which a web application runs under Linux, in order to reduce energy consumption while preserving quality of service. This research is conducted using an experimental and comparative approach, applied to two web application servers, Nginx and Node.js, in controlled and reproducible scenarios. Application performance (latency and throughput), energy consumption, and low-level metrics were measured using system profiling tools. The results show that, for Nginx, used with a predominantly I/O-oriented load, CPU affinity has a limited impact on performance and energy consumption. Conversely, for a computationally intensive Node.js application, controlled allocation to a smaller set of cores can, in some cases, simultaneously improve energy efficiency and performance. Conversely, frequent migrations between cores result in a degradation of observed metrics and an increase in energy consumption. These results show that the proposed method is able to reduce server energy consumption by 0.41% compared to using the default CPU affinity predefined by the Linux operating system, while maintaining the quality of service of the web application.

Keywords : Energy consumption, Performance, Web application, Linux, CPU affinity

TABLE DES MATIÈRES

DÉDICACE	I
REMERCIEMENTS	II
RÉSUMÉ	III
ABSTRACT	IV
TABLE DES MATIÈRES	V
TABLE DES FIGURES	VII
LISTE DES TABLEAUX	IX
ABRÉVIATIONS	X
GLOSSAIRE	XI
INTRODUCTION GÉNÉRALE	1
1 CONCEPTS GÉNÉRAUX ET ÉTAT DE L'ART	4
1.1 Concepts généraux	5
1.1.1 Application Web	5
1.1.2 Affinité CPU	6
1.1.3 Consommation énergétique dans un système d'exploitation Linux	10
1.2 État de l'art	12
1.2.1 Technique de réduction de l'énergie consommée : DVFS	12
1.2.2 Principe de fonctionnement de la DVFS	12
1.2.3 Ordonnancement : Politique de traitement des processus/threads dans le noyau Linux	14
1.2.4 Bilan et positionnement	18
2 MÉTHODOLOGIE DE RECHERCHE	19
2.1 Approche générale de la recherche	20

TABLE DES MATIERES

2.1.1	Démarche scientifique adoptée	20
2.1.2	Phase préparatoire	21
2.1.3	Protocoles expérimentaux	23
2.1.4	Méthodes d'analyse	25
2.1.5	Modélisation du cadre d'analyse	26
2.2	Fondements méthodologiques de la recherche	28
2.2.1	Justification du choix de l'approche	28
2.2.2	Cadre scientifique de la recherche	28
2.3	Bilan	29
3	IMPLÉMENTATION ET RÉSULTATS	30
3.1	Implémentation du protocole expérimental	31
3.1.1	Ressources matérielles	31
3.1.2	Environnements logiciels	32
3.2	Résultats expérimentaux	38
3.2.1	Résultats pour le serveur web Nginx	39
3.2.2	Résultats pour le serveur web Node.js	45
3.2.3	Implications pour les déploiements en production	52
3.3	Discussion	53
3.3.1	Evaluation des hypothèses	53
3.3.2	Limites méthodologiques	54
3.4	Bilan	55
	CONCLUSION GÉNÉRALE ET PERPECTIVES	56
	RÉFÉRENCES	58
	ANNEXES	61



TABLE DES FIGURES

1.1	Architecture client/serveur [6]	5
1.2	Affinité CPU sur 1 et sur tous les cœurs du processeur [9]	7
1.3	Rôle de la mémoire cache dans les systèmes [11]	8
1.4	Cache CPU : L1, L2, L3 [11]	8
1.5	Topologies des processeurs Core i3 et Xeon [12]	9
1.6	Exemple d'architecture du processeur	14
1.7	Disposition des threads (premier cas)	15
1.8	Disposition des threads (deuxième cas)	16
1.9	Disposition des threads (troisième cas)	16
2.1	Vue globale de la méthodologie expérimentale	21
2.2	Exécution de l'application web Nginx	22
2.3	Exécution de l'application web Node.js	22
3.1	Diagramme de séquences de l'application Nginx	34
3.2	Diagramme de séquences de l'application Nginx	36
3.3	Latence des requêtes par affinité CPU - Nginx	39
3.4	Débit des requêtes par affinité CPU - Nginx	40
3.5	Fréquence des cœurs du processeur avec pour affinité tous les cœurs - Nginx	41
3.6	Fréquence des cœurs du processeur avec pour affinité 1 cœur (CPU0) - Nginx	41
3.7	Fréquence des cœurs du processeur avec pour affinité 5 cœurs (CPU0-4) - Nginx	42
3.8	Fréquence des cœurs du processeur avec pour affinité 10 cœurs (CPU0-9) - Nginx	42
3.9	Fréquence des cœurs du processeur avec pour affinité 1 cœur différent toutes les secondes - Nginx	43
3.10	Energie consommée - Nginx	44
3.11	Gain énergétique en fonction des scénarios - Nginx	44
3.12	Latence des requêtes par affinité CPU - Node.js	46
3.13	Débit des requêtes par affinité CPU - Node.js	47
3.14	Fréquence des cœurs du processeur avec pour affinité tous les cœurs - Node.js	48
3.15	Fréquence des cœurs du processeur avec pour affinité 1 cœur (CPU0) - Node.js	48
3.16	Fréquence des cœurs du processeur avec pour affinité 5 cœurs (CPU0-4) - Node.js	49

3.17	Fréquence des cœurs du processeur avec pour affinité 10 cœurs (CPU0-9) - Node.js . .	50
3.18	Fréquence des cœurs du processeur avec pour affinité 1 cœur différent toutes les secondes - Node.js	50
3.19	Energie consommée par le serveur Node.js	51
3.20	Gain énergétique en fonction des scénarios - Node.js	52

LISTE DES TABLEAUX

1.1	Tableau de synthèse	11
2.1	Métriques suivies pendant l'exécution des applications web	23
3.1	Spécifications matérielles des machines utilisées	32
3.2	Tableau comparatif des serveurs web	35
3.3	Tableau comparatif des outils de génération des requêtes	37
3.4	Tableau comparatif des outils de profilage CPU et performances	38
3.5	Tableau comparatif des outils de mesure de la consommation énergétique	38
3.6	Tableau comparatif des outils d'analyse du trafic réseau	38
3.7	Tableau comparatif des résultats statistiques des latences	39
3.8	Tableau comparatif des résultats statistiques des débits	40
3.9	Tableau comparatif des résultats statistiques de l'énergie consommée	44
3.10	Indicateurs Système en fonction de l'affinité CPU - Nginx	45
3.11	Tableau comparatif des résultats statistiques des latences	46
3.12	Tableau comparatif des résultats statistiques des débits	47
3.13	Tableau comparatif des résultats statistiques de l'énergie consommée	51
3.14	Indicateurs Système en fonction de l'affinité CPU - NodeJS	52
3.15	Gains financiers (en FCFA) par an pour 100.000 serveurs - Nginx	53
3.16	Gains financiers (en FCFA) par an pour 100.000 serveurs - NodeJS	53

ABRÉVIATIONS

AB	Apache Benchmark
APP	Application
BSD	Berkeley Software Distribution
CFS	Completely Fair Scheduler
CPU	Central Processing Unit
DVFS	Dynamic Voltage and Frequency Scaling
E/S	Entrée/Sortie
ENSPY	École Nationale Supérieure Polytechnique de Yaoundé
HTTP	HyperText Transfer Protocol
RAM	Random Access Memory
RAPL	Running Average Power Limit
SOSP	Symposium on Operating Systems Principles
TCP	Transmission Control Protocol
UDP	User Datagram Protocol

GLOSSAIRE

Affinité CPU : mécanisme du système d'exploitation permettant de restreindre l'exécution d'un processus ou d'un thread à un ensemble spécifique de cœurs du processeur.

Application web : Logiciel applicatif accessible via un navigateur web et conçu pour interagir avec des utilisateurs à travers une interface web.

Asynchrone : adjectif faisant référence au fait d'orchestrer des actions de façon à ce qu'elles ne soient pas dépendantes du temps des autres.

Cache : mémoire informatique volatile de petite taille qui enregistre temporairement des copies de données provenant d'une autre source, comme le disque dur ou la mémoire principale.

Cœur : petit processeur intégré dans un gros processeur ou socket CPU qui effectue ou traite indépendamment toutes les tâches de calcul.

Computational : adjectif qualifiant tout ce qui est relatif au calcul, au traitement de l'information ou à l'utilisation de méthodes informatiques pour résoudre des problèmes, simuler des phénomènes ou modéliser des systèmes.

CPUfreq : gestionnaire du système Linux qui permet d'ajuster la vitesse d'horloge du processeur spontanément.

Débit : quantité totale de données qui peuvent être transférées sur le réseau pendant une période donnée.

Empreinte mémoire : ensemble des ressources mémoire consommées par un processus ou un système, incluant la mémoire active, réservée et partagée, à un instant donné ou sur une période d'exécution.

Latence : temps qui s'écoule entre le moment où la requête est envoyée par le client et le moment où il reçoit la réponse correspondante.

Localité du cache : propriété qu'un processus ou threads à accéder à des données ou instructions stockées dans la mémoire la plus proche.

Médiane : valeur qui sépare la moitié inférieure et la moitié supérieure des termes d'une série statistique quantitative ordonnée.

Migration CPU : déplacement des processus ou threads d'un processeur à un autre, souvent pour optimiser l'utilisation des processeurs au sein de la machine.

Multithread : caractéristique des systèmes d'exploitation permettant à un processeur d'exécuter plusieurs fils d'exécution, appelés threads, de manière quasi simultanée.

Ordonnancement : Organisation méthodique des processus ou threads au sein du processeur afin d'être traités.

Pile réseau du noyau Linux : ensemble des composants logiciels intégrés au noyau qui implémentent les protocoles de communication réseau et assurent la transmission, la réception et le routage des données entre les applications et le matériel réseau.

Processeur : encore appelé CPU, est la partie d'un ordinateur exécutant le traitement des instructions, constitué des organes de commande et de calcul.

Processus : programme en cours d'exécution au sein d'un environnement.

Profilage : ensemble des techniques qui servent à observer, mesurer et analyser le comportement d'un système ou d'un programme pendant son exécution.

Proxy inverse : serveur intermédiaire entre le périphérique d'un utilisateur et l'internet, en gérant les demandes de contenu web pour l'utilisateur.

Scalabilité : capacité à supporter une augmentation de charge (utilisateurs, données, requêtes, ressources) sans dégradation significative des performances, ou en maintenant un coût maîtrisé.

Serveur proxy : serveur intermédiaire chargé de router le trafic Internet entre les utilisateurs et les ressources en ligne.

Serveur web : Les serveurs web hébergent et fournissent des sites web, en traitant les demandes des clients et en servant les pages web aux utilisateurs.

Sobriété énergétique : réduction volontaire et organisée des consommations d'énergie.

Thread : encore appelé processus léger, fil d'exécution, fil d'instruction ou processus allégé, c'est un sous-processus opérant dans le contexte d'un processus principal et utilisant ses ressources.

Variation inhérente : fluctuation naturelle et inévitable des temps d'exécution et de l'utilisation des ressources, due au fonctionnement interne du système.

INTRODUCTION GÉNÉRALE

CONTEXTE

L'essor des technologies web et la croissance exponentielle du trafic en ligne ont conduit à une augmentation significative de la demande en ressources informatiques. D'après le rapport de The Shift Project publié en 2019, la consommation d'énergie liée aux usages numériques augmente de 9% par an [1]. Selon cette même organisation en 2024, le secteur serait responsable de près de 4% des émissions de gaz à effet de serre dans le monde, soit davantage que le transport aérien civil. Et d'ici à 2030, cette part pourrait atteindre 7% si l'évolution se fait sans contrôle[2], ce qui mettrait en mal l'objectif de neutralité carbone (zéro émission nette) fixé par les pays et acteurs non étatiques pour l'année 2050[3].

Ainsi, l'optimisation énergétique des systèmes informatiques est devenue une priorité, tant pour des raisons économiques qu'environnementales. Les systèmes d'exploitation modernes, en particulier Linux, intègrent des mécanismes avancés d'ordonnancement et de gestion des ressources matérielles. Cependant, la répartition par défaut des processus sur les différents cœurs du processeur n'est pas toujours optimale[4].

PROBLÉMATIQUE

Les mécanismes d'ordonnancement par défaut de Linux privilégient généralement la répartition équilibrée de la charge sur les différents cœurs afin de maximiser la réactivité et le parallélisme. Cette stratégie, bien qu'efficace en termes de performance, peut engendrer une consommation énergétique accrue en maintenant plusieurs cœurs actifs de manière simultanée. Or, dans de nombreux cas d'utilisation, il n'est pas nécessaire d'exploiter tous les cœurs disponibles pour assurer une qualité de service satisfaisante. La question centrale qui se pose est la suivante :

Problématique

Comment réduire la consommation énergétique des applications web exécutées sous Linux, tout en maintenant des performances acceptables, à l'aide d'une gestion adaptée de l'affinité CPU ?

HYPOTHÈSES

À la lumière de cette problématique présentée, ce travail repose sur deux hypothèses principales :

- ✱ Une affectation contrôlée et réfléchie des processus à un ensemble restreint de cœurs peut réduire la consommation énergétique globale du système.
- ✱ Une réduction de la consommation énergétique est possible sans dégrader significativement les performances perçues par l'utilisateur final (latence, débit de requêtes, disponibilité).

OBJECTIFS

L'objectif principal de ce travail est de :

Objectif

Démontrer expérimentalement qu'une gestion adéquate de l'affinité CPU peut contribuer à une réduction mesurable de la consommation énergétique des applications web tournant sur un noyau Linux, sans dégrader leurs performances.

L'atteinte de cet objectif passe donc par les étapes suivantes :

- ✍ Analyser le comportement énergétique et les performances d'une application web selon différents scénarios d'affinité CPU.
- ✍ Mettre en évidence la relation entre la distribution des processus et la consommation énergétique du système.
- ✍ Identifier des stratégies d'affinité CPU permettant de concilier efficacité énergétique et qualité de service.

METHODOLOGIE

Pour mieux cerner l'impact de l'affinité CPU sur la consommation énergétique, nous avons élaboré une méthodologie expérimentale et comparative, reposant sur l'observation d'exécutions d'applications web dans un environnement Linux. Cette approche se décline en deux grandes étapes complémentaires :

- ✱ **Phase expérimentale avec profilage** : elle est menée afin d'observer l'impact de différentes stratégies d'affinité CPU sur la consommation énergétique d'une application web.

Pour ce faire, un environnement de test basé sur Linux est mis en place, incluant une application web et un générateur de charge. Plusieurs scénarios d'affinité CPU sont définis : une répartition libre des processus par l'ordonnanceur et des affinités restreintes à des sous-ensembles de cœurs. Durant ces expérimentations, des outils de profilage seront mobilisés afin de mesurer avec précision la consommation énergétique, l'activité des processeurs, le comportement du système sous charge et les performances de l'application.

- ❄ **Analyse comparative des métriques collectées** : elle est menée afin d'évaluer objectivement les effets des différentes configurations testées. Les données recueillies (consommation énergétique, nombre de migrations CPU, débit de requêtes, etc.) seront structurées et comparées à l'aide de tableaux et de représentations graphiques. Cette analyse permettra d'identifier les scénarios dans lesquels l'affinité CPU favorise une meilleure efficacité énergétique tout en maintenant des performances satisfaisantes. L'objectif est ainsi de dégager des stratégies de gestion des ressources offrant le meilleur compromis entre performance et sobriété énergétique, ouvrant la voie à des recommandations concrètes pour des environnements de production.

PLAN DU MÉMOIRE

La suite de ce mémoire est organisé en trois chapitres principaux, chacun correspondant à une étape clé du travail réalisé.

- ❄ **Chapitre 1 : Concepts généraux et État de l'art** : Ce chapitre introduit les notions fondamentales nécessaires à la compréhension du sujet, en posant les bases théoriques et techniques de la recherche. Ensuite une revue approfondie de la littérature existante est présentée afin d'examiner les approches, méthodes et outils déjà mis en place.
- ❄ **Chapitre 2 : Méthodologie de recherche** : Ce chapitre expose la méthodologie adoptée pour la formalisation du système. Il détaille l'analyse préalable, les procédures expérimentales et la modélisation mise en œuvre, en insistant sur les métriques déterminantes de la consommation énergétique.
- ❄ **Chapitre 3 : Implémentation et résultats** : Ce chapitre décrit l'implémentation des protocoles expérimentaux, en précisant les outils mobilisés, puis présente les résultats obtenus. Il en propose une analyse approfondie et situe la solution développée par rapport aux approches existantes, tout en soulignant la continuité logique de nos travaux.

Le mémoire se conclut par une synthèse générale rappelant le contexte, les objectifs et la méthodologie, faisant le bilan des travaux accomplis, soulignant l'intérêt de la démarche et proposant des pistes pour des travaux futurs.

CONCEPTS GÉNÉRAUX ET ÉTAT DE L'ART

C *e chapitre introduit les notions fondamentales nécessaires à la compréhension du sujet, en posant les bases théoriques et techniques de la recherche. Ensuite une revue approfondie de la littérature existante est présentée afin d'examiner les approches, méthodes et outils déjà mis en place.*

1.1 CONCEPTS GÉNÉRAUX

Cette première partie introduit les notions fondamentales nécessaires à la compréhension du travail. Elle présente successivement le fonctionnement des applications web, le concept d'affinité CPU, puis les mécanismes liés à la consommation énergétique dans un système d'exploitation Linux.

1.1.1 Application Web

Les applications web constituent aujourd'hui l'une des charges de travail les plus répandues dans les environnements serveurs, notamment au sein des infrastructures cloud et des centres de données [5].

1.1.1.1 Définition, architecture et avantages

Une application web est un logiciel d'application qui s'exécute sur un serveur Web. L'application Web est accessible par le client via un navigateur Web avec une connexion Internet active pour effectuer des requêtes. Avec une architecture client-serveur, elle met en jeu deux composants minimum :

- **Le client** : c'est l'utilisateur qui interagit avec l'application et envoie des requêtes.
- **Le serveur** : il héberge les modules logiciels de l'application et gère les requêtes des utilisateurs.

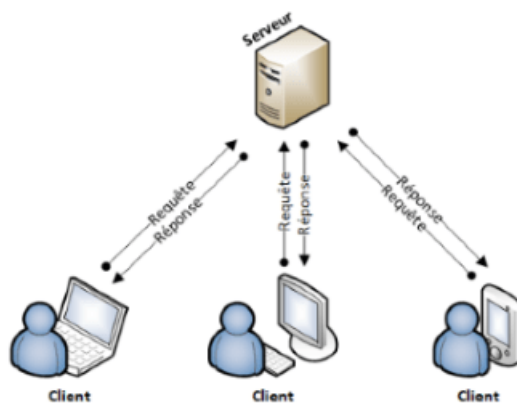


FIGURE 1.1 – Architecture client/serveur [6]

Comme avantages les plus courants des applications web, nous pouvons citer :

- **Accessibilité** : les applications Web sont accessibles depuis n'importe quel navigateur Web et depuis de nombreux appareils personnels et professionnels. Les équipes travaillant dans des endroits différents sont en mesure d'accéder à des documents partagés, des systèmes de gestion de contenu et d'autres services professionnels grâce aux applications Web [7].

- **Simplicité d'utilisation** : les applications Web n'ont pas besoin d'être téléchargées, ce qui facilite leur accès tout en éliminant les exigences de maintenance et d'espace libre sur le disque dur de l'utilisateur final. Les applications Web reçoivent automatiquement les mises à jour de sécurité logicielles ; elles sont ainsi constamment à jour et présentent moins de failles de sécurité dans ce sens là.

1.1.1.2 Fonctionnement des applications Web

Dans la technologie client-serveur, le client à travers un navigateur Web envoie au serveur des requêtes relatives à des pages Web. Le serveur répond aux demandes en envoyant des réponses au navigateur Web. Le navigateur affiche alors les pages à l'utilisateur. Les applications Web utilisent cette technique pour mettre en œuvre leur interface graphique. Celle-ci est composée de pages créées de toutes pièces par le logiciel lors de chaque requête. Chaque hyperlien contenu dans la page provoque l'envoi d'une nouvelle requête, qui donnera en résultat une nouvelle page. À la différence d'un site web statique où les pages sont des fichiers préalablement enregistrés.

Pendant son fonctionnement, une application web peut être soumise à des charges hautement variables : les requêtes arrivent de manière asynchrone, entraînant des fois, des pics soudains d'activité. Dans ce contexte, la performance de traitement dépend directement du temps CPU attribué à chaque requête et de la rapidité du cœur sur lequel elle s'exécute. Ainsi, à charge équivalente, les performances d'une application web peuvent varier significativement selon les décisions prises par le système d'exploitation. Les systèmes d'exploitation jouent ainsi un rôle clé dans la distribution des processus générés par les requêtes sur les différents cœurs du CPU. Et le comportement de l'ordonnanceur influence directement la latence perçue par les utilisateurs. C'est dans ce cadre qu'une bonne distribution de ces processus au niveau des cœurs est nécessaire.

1.1.2 Affinité CPU

1.1.2.1 Définition et principe

L'**affinité CPU** est un mécanisme qui permet d'affecter des processus à des cœurs spécifiques, ce qui peut améliorer les performances en réduisant les changements de contexte et en optimisant l'utilisation du cache [8]. Par défaut, l'ordonnanceur du noyau Linux est libre de planifier un processus sur n'importe quel cœur disponible, dans le but de maximiser l'équité et l'utilisation globale du processeur. L'affinité CPU permet de limiter cette liberté en associant à chaque processus un masque de cœurs, définissant les unités de calcul sur lesquelles il est autorisé à s'exécuter. Ce mécanisme ne modifie pas l'ordonnancement interne du noyau, mais contraint l'espace de décision de l'ordonnanceur. Il s'agit ainsi d'un levier de contrôle fin, non intrusif, pouvant être appliqué dynamiquement sans modification du noyau ou du matériel.

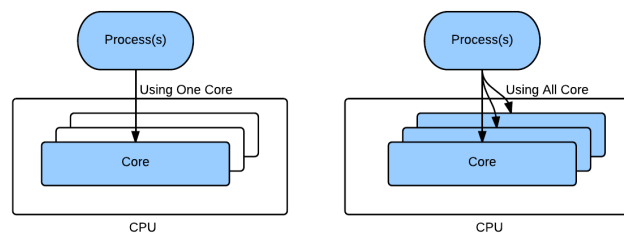


FIGURE 1.2 – Affinité CPU sur 1 et sur tous les cœurs du processeur [9]

L'**épinglage CPU** est une forme plus stricte d'affinité CPU, où un processus ou un thread est lié à un seul cœur de CPU spécifique, et donc il n'est pas possible pour lui de s'exécuter sur un autre cœur. Cela signifie que toute possibilité de migration de processus est éliminée. L'épinglage du processeur se fait généralement au niveau du noyau, en modifiant le comportement du planificateur ou en utilisant des outils ou des bibliothèques spéciaux. Dans la plupart des systèmes d'exploitation, l'ensemble des processeurs sur lesquels un processus ou un thread est autorisé à s'exécuter est exprimé sous la forme d'un masque d'affinité, qui est un masque binaire correspondant aux cœurs du système.

1.1.2.2 Implémentation de l'affinité CPU sous Linux

Sous Linux, l'affinité CPU est implémentée au niveau du noyau via des structures de données associées aux processus et aux threads. Elle peut être configurée : par des appels système ou par des outils en espace utilisateur tels que Taskset [10]. Lorsqu'un processus est soumis à une affinité, l'ordonnanceur s'assure qu'il ne sera exécuté que sur ces cœurs, même si d'autres cœurs sont disponibles.

L'affinité CPU est représentée par un masque de bits, le bit de poids faible correspondant au premier cœur CPU logique et le bit de poids fort au dernier. Un masque récupéré ne reflétera que les bits qui correspondent aux processeurs logiques présents sur le système. Les masques peuvent être spécifiés en hexadécimal (avec ou sans « 0x » en préfixe), ou sous forme de liste de processeurs. Par exemple :

- **0x00000001** représente le processeur #0.
- **0x00000003** représente les processeurs #0 et #1.
- **FFFFFFFF** représente les processeurs de #0 à #31.
- **0x32** représente les processeurs #1, #4 et #5.

1.1.2.3 Affinité CPU et localité des caches

La **mémoire cache** est un composant essentiel des systèmes modernes, introduit pour combler l'écart grandissant entre la rapidité des processeurs et la relative lenteur de la mémoire

principale. Sans ce mécanisme, les performances des processeurs resteraient limitées par la fréquence des accès à la RAM, créant ainsi un goulet d'étranglement. Le cache agit comme une mémoire intermédiaire ultra-rapide, capable de fournir aux unités de calcul les données et instructions les plus sollicitées [11] (Voir Figure 1.3).

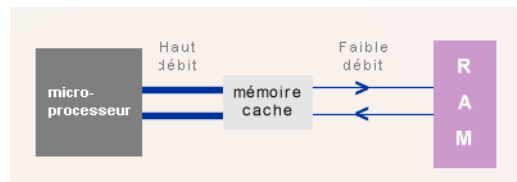


FIGURE 1.3 – Rôle de la mémoire cache dans les systèmes [11]

La **mémoire cache CPU** constitue un élément essentiel de la hiérarchie mémoire des processeurs modernes. Elle agit comme une zone intermédiaire entre le processeur et la mémoire principale, permettant d'accélérer considérablement l'accès aux données et instructions les plus sollicitées. Pour optimiser cet équilibre entre vitesse et capacité, elle est organisée en plusieurs niveaux hiérarchisés (L1, L2 et L3), chacun contribuant à améliorer la fluidité et l'efficacité du traitement, comme présenté dans la Figure 1.4 ci-dessous. Cette organisation hiérarchique permet de combiner la rapidité des caches les plus proches du processeur avec la capacité plus importante des caches de niveaux supérieurs. Ainsi, le cache CPU joue un rôle déterminant dans la réduction de la latence et l'optimisation globale des performances des systèmes informatiques modernes.

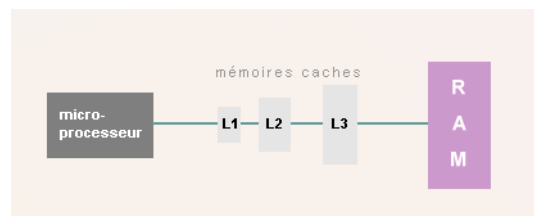


FIGURE 1.4 – Cache CPU : L1, L2, L3 [11]

Le **cache L1**, situé au plus près du processeur, constitue la mémoire la plus rapide grâce à son temps d'accès extrêmement faible, mais sa capacité reste très limitée. Le **cache L2**, situé entre le L1 et le L3, offre une capacité supérieure à celle du L1 mais avec un accès légèrement plus lent. Le **cache L3**, pour sa part, est partagé entre tous les cœurs du processeur et possède une capacité bien plus importante. Il favorise la cohérence et la scalabilité entre cœurs. La figure 1.5 nous présente deux exemples de disposition et partage des mémoires cache au sein du processeur.

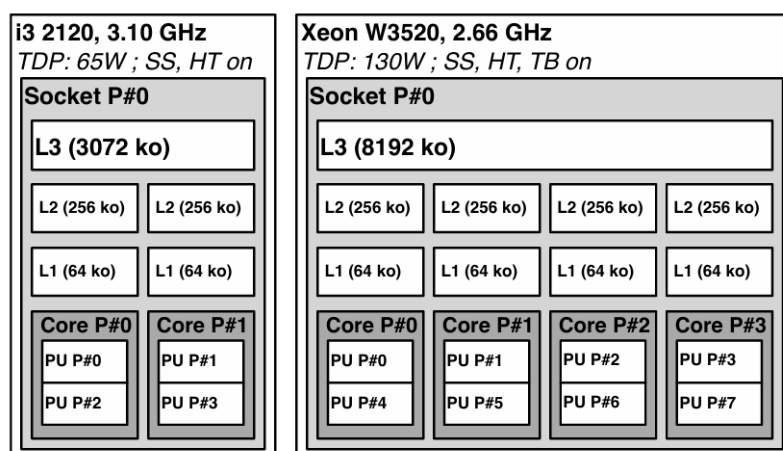


FIGURE 1.5 – Topologies des processeurs Core i3 et Xeon [12]

Ainsi, l'un des effets majeurs de l'affinité CPU concerne la localité des caches. En restreignant l'exécution d'un processus à un nombre limité de cœurs, il devient possible de conserver plus efficacement ses données et instructions dans les caches CPU. À l'inverse, les migrations fréquentes de processus entre cœurs pourraient entraîner des invalidations de cache et une augmentation des accès à la mémoire principale, ce qui dégrade les performances. L'affinité CPU permet ainsi de stabiliser l'exécution et de réduire ces surcoûts.

1.1.2.4 Avantages

Les avantages de l'utilisation de l'affinité du processeur sont les suivantes :

- **Amélioration des performances** : en réduisant les migrations et en améliorant la localité des données, l'affinité du processeur peut conduire à de meilleures performances du cache et à une latence d'accès à la mémoire plus faible. En effet, quand un processus migre d'un cœur à un autre, les caches L1, L2 ou L3 sont perdus, par la suite il faudra les recharger, ce qui coûte du temps et de l'énergie. Ainsi, l'affinité CPU réduit les migrations inter-cœurs, améliore la localité cache, diminue le temps de latence.
- **Stabilité et prévisibilité** : l'attribution de processus critiques à des cœurs dédiés peut améliorer la fiabilité et la prévisibilité du système, notamment dans les environnements de calcul haute performance. En fixant un processus sur un cœur, le comportement devient plus déterministe, on réduit la variabilité des temps d'exécution.
- **Gestion des ressources** : L'affinité du processeur impose également une répartition statique des ressources de traitement. Elle permet ainsi de limiter le nombre de cœurs CPU utilisés par un processus gourmand en ressources CPU, laissant ainsi les autres cœurs disponibles pour les autres programmes. Cette approche n'est évidemment pas optimale, car elle laisserait des ressources totalement inutilisées lorsqu'aucun autre programme n'est en cours d'exécution, tout en permettant à d'autres programmes de concurrencer le

programme gourmand en ressources CPU pour les ressources sur les quelques cœurs autorisés. Des méthodes plus avancées de répartition des ressources incluent les paramètres de priorité CPU, les parts d'utilisation CPU et les limites strictes de pourcentage d'utilisation.

1.1.2.5 Limites et compromis

Comme inconvénients de l'affinité du processeur, nous pouvons citer :

- **Flexibilité réduite** : Limiter un processus à des cœurs spécifiques peut diminuer l'adaptabilité du système aux variations de charge et entraîner une sous-utilisation des ressources disponibles.
- **Complexité de la gestion** : La surveillance et l'ajustement des paramètres d'affinité du processeur peuvent accroître la complexité de la gestion du système, notamment dans les environnements dynamiques.
- **Risque de dégradation des performances** : Une configuration d'affinité trop stricte, si elle n'est pas gérée avec précaution, peut engendrer des problèmes tels qu'une latence accrue ou des goulots d'étranglement si le cœur attribué est surchargé ou rencontre des problèmes matériels. Toutefois, une affinité trop rigide peut créer des déséquilibres : certains cœurs peuvent être surchargés alors que d'autres demeurent sous-utilisés. Il est donc essentiel d'adapter dynamiquement cette affinité en fonction de la charge et des caractéristiques fréquentielles des cœurs.

1.1.3 Consommation énergétique dans un système d'exploitation Linux

La consommation énergétique d'un système informatique ne dépend pas uniquement des caractéristiques matérielles, mais résulte d'une interaction étroite entre le matériel et le système d'exploitation. Dans les systèmes modernes, et en particulier sous Linux, le noyau joue un rôle central dans la gestion de l'énergie, en contrôlant l'utilisation des ressources matérielles, l'ordonnancement des tâches et les transitions entre différents états de fonctionnement des composants.

Bien que la consommation énergétique d'un système informatique dépende en grande partie des caractéristiques matérielles du processeur et des composants associés, elle ne peut être considérée comme un phénomène purement matériel. En effet, les processeurs multicœurs modernes offrent de multiples mécanismes de gestion de l'énergie, notamment la variation dynamique de fréquence et de tension, les états d'inactivité ou le parallélisme matériel, dont l'activation effective est pilotée par le système d'exploitation. Par ses décisions d'ordonnancement, de gestion des threads et de contrôle des états de puissance, le système d'exploitation influence directement le nombre de cœurs actifs, leur fréquence de fonctionnement et la durée passée dans les états haute et basse consommation. Ainsi, avec une charge applicative identique, des politiques logicielles différentes peuvent conduire à des consommations énergétiques significativement distinctes, faisant du système d'exploitation un acteur clé de l'efficacité énergétique globale.



1.1.3.1 Facteurs influençant la consommation énergétique

D'après l'article « A study on factors influencing power consumption in multithreaded and multicore CPUs » de Saravanan Vijayalakshmi et Al. [13], les paramètres matériels et logiciels qui influencent la consommation énergétique des processeurs modernes sont les suivants :

- **Nombre de cœurs actifs** La consommation augmente significativement avec le nombre de cœurs actifs, même à charge constante.
- **Multithreading et parallélisme** augmenter le nombre de threads n'augmente pas linéairement les performances, mais augmente presque toujours la consommation.

1.1.3.2 Rôle du système d'exploitation Linux

Le noyau Linux agit comme un intermédiaire entre les applications et le matériel. Il influence la consommation énergétique à travers plusieurs mécanismes :

- la planification et le placement des processus.
- la gestion de la fréquence et de la tension du processeur.
- le contrôle des états d'inactivité des cœurs.
- la gestion de la mémoire et des accès d'entrée/sortie.

Ces décisions déterminent le nombre de cœurs actifs, leur niveau d'activité et la durée pendant laquelle ils restent dans des états de consommation élevée. Le tableau suivant présente les différents composants du système d'exploitation et les paramètres qu'ils affectent.

TABLE 1.1 – Tableau de synthèse

Facteur	Mécanisme Linux associé
Nombre de cœurs actifs	Scheduler
Multithreading	Gestion des threads
Fréquence CPU	Gouverneurs
Temps d'inactivité	C-states
Migrations	Affinité CPU

1.1.3.3 Gestion de la fréquence et des états d'alimentation du CPU

Linux intègre des sous-systèmes dédiés à la gestion de l'énergie du processeur. Le mécanisme de CPU frequency scaling (CPUfreq) permet d'adapter dynamiquement la fréquence et la tension des cœurs en fonction de la charge, en s'appuyant sur différents gouverneurs (performance, powersave, ondemand, schedutil).

En parallèle, la gestion des états de repos (C-states) permet aux cœurs inactifs d'entrer dans des niveaux de veille plus ou moins profonds. Plus l'état est profond, plus la consommation est

faible, mais plus le coût de réveil est élevé. Le système d'exploitation doit donc arbitrer entre réactivité et économie d'énergie.

1.1.3.4 Mesure et observation de la consommation énergétique sous Linux

Linux fournit plusieurs interfaces permettant d'observer et d'estimer la consommation énergétique. Les compteurs matériels accessibles via des mécanismes comme RAPL (Running Average Power Limit) pour les processeurs Intel permettent de mesurer la consommation du processeur et de la mémoire avec une granularité fine. Des outils tels que powerstat, perf ou turbostat exploitent ces interfaces pour fournir des métriques énergétiques exploitables. Ces mécanismes sont essentiels pour analyser l'impact des politiques d'ordonnancement et de placement sur la consommation énergétique.

1.2 ÉTAT DE L'ART

1.2.1 Technique de réduction de l'énergie consommée : DVFS

La mise à l'échelle dynamique de la tension et de la fréquence (DVFS) est une technique largement utilisée pour réduire la consommation d'énergie des processeurs en faisant varier la tension et la fréquence en cours d'exécution [14]. L'idée principale est de réduire la fréquence ou la tension pendant les périodes où la charge de travail du processeur est réduite. Si la DVFS est mise en œuvre judicieusement, il est possible d'économiser de l'énergie sans incidence notable sur la vitesse d'exécution du processeur. La plupart des systèmes sont conçus avec des paramètres de tension et de fréquence fixes afin d'assurer leur stabilité. Cependant, les niveaux d'activité des applications sont variables et, dans de nombreux cas, elles connaissent des périodes d'inactivité durant lesquelles aucune tâche utile n'est effectuée. En réduisant les niveaux de tension et de fréquence du processeur en cours d'exécution lorsque l'application présente une faible activité ou des périodes d'inactivité, il est possible d'économiser de l'énergie.

1.2.2 Principe de fonctionnement de la DVFS

Le gestionnaire de la fréquence du processeur (CPUFreq) est implémentée dans le noyau Linux pour contrôler la fréquence du processeur, fonctionnant comme un pilote de périphérique. Le fichier cpufreq du noyau Linux permet de visualiser et de modifier la fréquence de chaque cœur du processeur, situé à l'emplacement suivant : `/sys/devices/system/cpu/cpux`. Différentes politiques de gouvernance pour CPUFreq répondent à divers besoins de performance. On distingue plusieurs gouverneurs parmi lesquels Performance, PowerSave, Conservative, OnDemand, Userspace et Schedutil.

La DVFS exploite cette propriété en ajustant dynamiquement les états de performance du processeur, en fonction de l'activité observée. Lorsque la charge est faible, le processeur fonctionne

à une fréquence et une tension réduites, ce qui permet de diminuer la consommation énergétique. À l'inverse, lorsque la charge augmente, le processeur peut revenir à des états de performance plus élevés pour garantir les performances requises.

1.2.2.1 DVFS dans les systèmes d'exploitation modernes

Dans les systèmes d'exploitation modernes, et en particulier sous Linux, la DVFS est gérée par le noyau à travers des gouverneurs de fréquence CPU. Ces gouverneurs déterminent la politique d'adaptation de la fréquence en fonction de différents critères, tels que équilibrer l'efficacité énergétique et les performances.

Parmi les gouverneurs les plus courants, on trouve :

- **Performance** : il maintient le processeur à sa fréquence maximale dans les limites de fréquence minimales et maximales définies afin de garantir des performances optimales.
- **Conservative** : il fait fonctionner le processeur à la fréquence la plus basse dans la même plage afin de privilégier les économies d'énergie.
- **Userspace** : il permet aux utilisateurs ou aux programmes disposant des privilèges root de définir manuellement la fréquence du processeur via l'interface sysfs [15].
- **Schedutil** : il ajuste dynamiquement la fréquence du processeur en fonction de la charge système, les changements de fréquence étant déclenchés par le mécanisme d'estimation de charge du planificateur [16].

Ces gestionnaires permettent au système de gérer les performances du processeur et la consommation d'énergie en fonction des variations de la charge de travail.

1.2.2.2 Avantages et limites de la DVFS

La DVFS présente plusieurs avantages majeurs. Elle permet une réduction significative de la consommation énergétique et de la dissipation thermique, tout en étant transparente pour les applications. Elle est particulièrement efficace pour les charges de travail variables, où les périodes de faible activité peuvent être exploitées pour économiser de l'énergie.

Cependant, la DVFS présente également certaines limites. Tout d'abord, la réduction de la fréquence peut entraîner une augmentation du temps d'exécution, ce qui peut, dans certains cas, compenser les gains énergétiques réalisés. De plus, les transitions fréquentes entre états de fréquence peuvent induire un surcoût et une instabilité des performances, en particulier pour les applications sensibles à la latence. Ainsi, la DVFS agit principalement au niveau du processeur et ne prend pas directement en compte la nature des applications ni leur comportement en termes de localité mémoire ou de parallélisme.

1.2.3 Ordonnancement : Politique de traitement des processus/threads dans le noyau Linux

1.2.3.1 Rôle de l'ordonnanceur dans les systèmes multicœurs

L'ordonnanceur du noyau Linux est responsable de la sélection des tâches à exécuter et de leur affectation aux cœurs disponibles. Dans un contexte multicœur, il doit concilier plusieurs objectifs : garantir l'équité entre les processus, réduire la latence et limiter la consommation énergétique.

Un ordonnancement inadapté peut entraîner des migrations fréquentes de processus entre cœurs, provoquant une augmentation des coûts liés aux changements de contexte. Ces migrations ont également un impact énergétique non négligeable, car elles peuvent forcer l'activation de cœurs qui auraient pu rester inactifs ou dans des états de faible consommation.

1.2.3.2 Types de planification des threads

Certains types d'applications à charge inégale peuvent subir une dégradation importante de leurs performances. Ceci est dû au fait que l'ordonnanceur Linux traite le code des applications à hautes performances de la même manière qu'un processus système qui passe la majeure partie de son temps inactif [17].

Ces problèmes d'ordonnancement sont plus facilement compréhensibles à l'aide de schémas. Prenons l'exemple de nœuds de calcul dotés de deux sockets pour processeurs, chaque processeur possédant quatre cœurs :

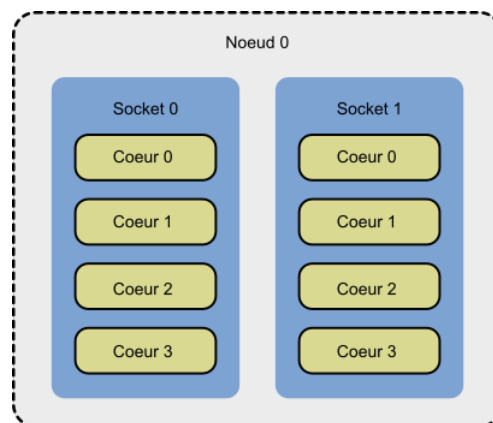


FIGURE 1.6 – Exemple d'architecture du processeur

Lorsqu'une application multithreadée à quatre threads (ou même quatre applications séquentielles) est exécutée, Linux planifie l'exécution de ces threads en les assignant chacun à un cœur du processeur. Sans indication explicite de la méthode de planification, Linux peut décider de :

- **Option n° 1** : exécuter les threads 0 à 3 sur les cœurs 0 à 3 du socket 0 ;

- **Option n° 2** : exécuter les threads 0 et 1 sur les cœurs 0 et 1 du socket 0, et les threads 2 et 3 sur le socket 1 ;
- **Option n° 3** : exécuter les threads 0 et 1 uniquement sur le cœur 0, le thread 2 sur le cœur 1, le thread 3 sur le cœur 2, et laisser le cœur 3 inutilisé ;
- **Option n° 4** : assigner plusieurs threads à un seul cœur tandis que les autres restent inactifs.

Dans cet exemple, les options 3 et 4 sont mauvaises pour les performances, mais Linux planifiera sans problème votre tâche multithreadée (ou plusieurs tâches monothreadées) de cette manière si le comportement de vos threads est source de confusion pour le système d'exploitation.

a) Ordonnancement compact

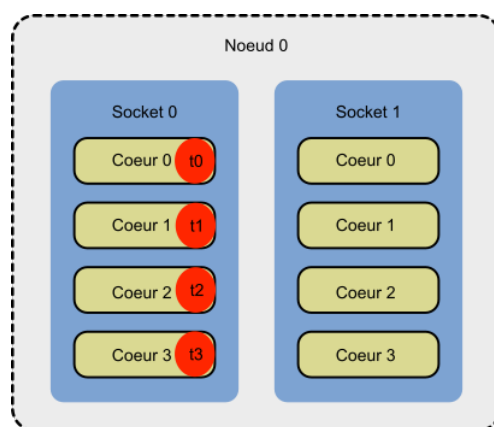


FIGURE 1.7 – Disposition des threads (premier cas)

L'option n° 1, souvent appelée ordonnancement « compact » illustrée dans la figure 1.7, consiste à exécuter tous les threads sur un seul processeur physique, si possible. C'est la solution idéale si tous les threads de votre application doivent accéder de manière répétée à données communes mises en cache. En effet, tous les cœurs d'un même processeur physique peuvent accéder aux banques de mémoire qui lui sont associées à la même vitesse. Cependant, les cœurs ne peuvent pas accéder aussi rapidement à la mémoire stockée sur des banques de mémoire appartenant à un autre processeur ; ce phénomène est appelé NUMA (accès non uniforme à la mémoire). Si tous vos threads doivent accéder à des données stockées dans la mémoire appartenant à un seul processeur, il est généralement préférable de les exécuter sur ce processeur.

b) Ordonnancement Round-Robin

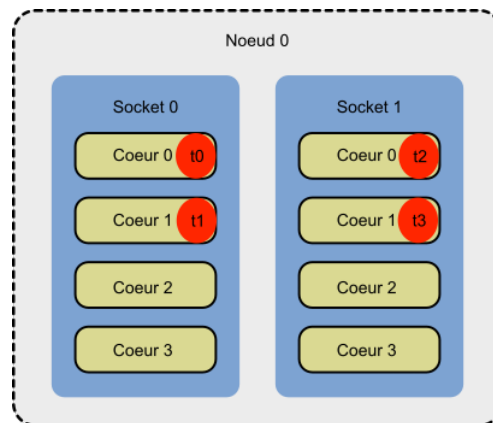


FIGURE 1.8 – Disposition des threads (deuxième cas)

L'option n° 2, appelée ordonnancement dispersé ou Round-Robin et illustrée par la figure 1.8, est idéale si vos threads sont largement indépendants les uns des autres et n'ont pas besoin d'accéder à une grande quantité de mémoire partagée par d'autres threads. L'avantage de l'ordonnancement Round-Robin est que tous les threads n'ont pas à partager le même canal mémoire et le même cache, ce qui double la bande passante mémoire et la taille du cache disponibles pour votre application.

c) Ordonnancement inadapté

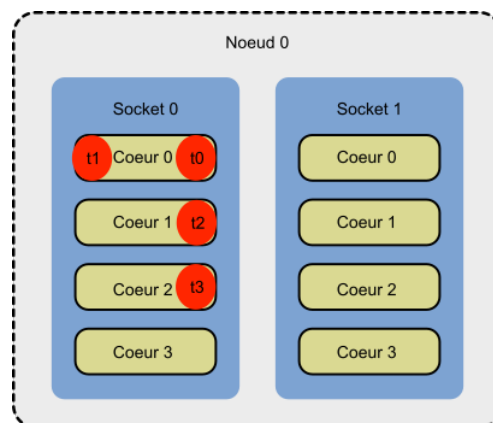


FIGURE 1.9 – Disposition des threads (troisième cas)

Les options 3 et 4 correspondent, dans notre exemple, à un ordonnancement « inadapté » (voir figure 1.9). Il s'agit souvent du comportement par défaut du planificateur de threads Linux si vous n'indiquez pas à Linux où vos threads doivent s'exécuter. Cela se produit car, dans les environnements serveurs Linux traditionnels, la plupart des processus en cours d'exécution sont inactifs. Afin d'économiser de l'énergie, Linux regroupe ces processus inactifs sur le même processeur ou les mêmes cœurs, puis les déplace vers un cœur dédié lorsqu'ils sont réactivés et doivent démarrer leur traitement. Le problème pour notre cas

d'étude est que si une application fonctionne à plein régime en permanence, Linux la maintiendra probablement sur son cœur de processeur dédié. Cependant, si la charge de votre application est inégale (par exemple, si la plupart des threads sont inactifs pendant que le dernier se termine), Linux constatera que l'application est majoritairement inactive et regroupera tous les threads inactifs (par exemple, t0 et t1 dans la figure 1.9) sur le même cœur de processeur. Ce ne serait pas si grave, mais le coût du déplacement d'un thread d'un cœur à un autre nécessite des changements de contexte qui deviennent très coûteux lorsqu'ils sont effectués des centaines ou des milliers de fois par minute [17].

1.2.3.3 Politique de placement, migrations et réveil des cœurs

Le noyau Linux utilise diverses stratégies pour déterminer le placement initial des processus et décider de leur migration éventuelle. Ces décisions prennent en compte la **charge des cœurs**, la **priorité des tâches** et la **topologie matérielle**. Toutefois, un ordonnancement trop agressif peut entraîner le réveil inutile de cœurs précédemment inactifs. En effet, les processeurs prennent en charge plusieurs technologies leur permettant d'optimiser la consommation d'énergie à l'instar des états P (optimisation de la tension et de la fréquence du processeur en fonctionnement) et des états C (optimisation de la consommation d'énergie lorsqu'un cœur n'exécute aucune instruction) [18]. Lorsqu'un processus est déplacé vers un cœur endormi, ce cœur doit sortir d'un état de faible consommation (C-state profond), ce qui entraîne un surcoût énergétique immédiat. De plus, la multiplication des cœurs actifs augmente la consommation statique et dynamique du processeur [13].

Ainsi, même si la charge globale reste identique, une mauvaise politique de placement peut augmenter la consommation énergétique en dispersant l'exécution sur un grand nombre de cœurs au lieu de la concentrer sur un sous-ensemble restreint.

1.2.3.4 Architectures NUMA et implications sur l'ordonnancement

Les architectures NUMA (Non-Uniform Memory Access) ajoutent une complexité supplémentaire à la problématique de l'ordonnancement. Dans ces systèmes, la latence d'accès à la mémoire dépend du nœud NUMA auquel appartient le cœur exécutant le processus. Un accès à la mémoire locale est plus rapide et moins coûteux énergétiquement qu'un accès à une mémoire distante.

Un mauvais placement des processus dans un environnement NUMA peut conduire à des accès mémoire distants fréquents, augmentant la latence d'exécution et la consommation énergétique. De plus, la migration d'un processus entre nœuds NUMA peut entraîner le réveil de cœurs supplémentaires et la perte de la localité mémoire précédemment établie.

Le noyau Linux intègre des mécanismes de NUMA balancing visant à rapprocher les processus de leurs données. Cependant, ces mécanismes peuvent entrer en conflit avec des objectifs

de stabilité ou d'efficacité énergétique, notamment dans des charges applicatives sensibles à la latence.

1.2.3.5 Ordonnancement, NUMA et efficacité énergétique

Dans un contexte multicœur et NUMA, la consommation énergétique dépend non seulement du nombre de cœurs actifs, mais aussi de leur localisation et de leurs interactions avec la mémoire. Une dispersion excessive des tâches peut activer plusieurs nœuds NUMA et multiplier les accès distants, augmentant ainsi la consommation globale.

À l'inverse, une politique de placement plus restrictive permet de :

- limiter le nombre de cœurs et de nœuds NUMA actifs ;
- préserver la localité des caches et de mémoire ;
- réduire les transitions d'état d'alimentation des cœurs.

Ces observations soulignent l'importance d'un contrôle explicite du placement des processus pour améliorer l'efficacité énergétique et les performances.

1.2.4 Bilan et positionnement

1.2.4.1 Bilan

Les travaux existants sur la réduction de la consommation énergétique et l'amélioration des performances des systèmes informatiques s'appuient principalement sur la DVFS et l'ordonnancement du système d'exploitation. Si la DVFS permet des gains significatifs d'énergie, son efficacité dépend fortement des décisions d'ordonnancement du noyau, en particulier lorsqu'il s'agit des migrations de processus et du réveil des cœurs. Ainsi dans un souci d'équilibrage des charges au sein des processeurs, une distribution maladroite des processus pourrait entraîner une augmentation des migrations des processus, suivie d'un accès aux mémoires cache plus distant et engendrer des réveils inappropriés des cœurs, imputant des coûts supplémentaires d'énergie consommée.

1.2.4.2 Positionnement

Ce travail se positionne comme une approche complémentaire et non intrusive, visant à exploiter l'affinité CPU, mécanisme natif du noyau Linux, pour améliorer conjointement performances et consommation énergétique des applications web. À travers une analyse expérimentale sur certaines applications web, il évalue l'impact de la concentration de l'exécution sur un ensemble de cœurs réduit et l'impact des migrations forcées sur l'efficacité énergétique et la qualité de service. Ce positionnement intermédiaire permet de montrer que des choix logiciels simples au niveau du système d'exploitation peuvent constituer un levier pertinent pour réduire l'empreinte énergétique des serveurs web.



MÉTHODOLOGIE DE RECHERCHE

Ce chapitre décrit la méthodologie adoptée dans le cadre de cette étude. Il détaille la manière dont nous avons évalué l'impact de la mise en place de l'affinité CPU sur l'exécution d'une application web sous Linux, en veillant à garantir la rigueur scientifique et la reproductibilité des expérimentations. Les protocoles mis en œuvre permettent de mesurer conjointement les performances applicatives (temps de réponse et débit) ainsi que la consommation énergétique du système. Enfin, ce chapitre présente les méthodes d'analyse utilisées pour interpréter les résultats obtenus, ainsi que les limites de l'approche proposée.



2.1 APPROCHE GÉNÉRALE DE LA RECHERCHE

La recherche adopte une approche expérimentale comparative, complétée par une modélisation analytique, visant à évaluer l'impact de l'affinité CPU sur les performances et la consommation énergétique d'une application web sous Linux.

2.1.1 Démarche scientifique adoptée

La méthode choisie repose sur une approche expérimentale et comparative visant à analyser l'impact de la configuration de l'affinité CPU sur les performances et la consommation énergétique de serveurs web exécutés sous Linux. Elle consiste à exécuter plusieurs serveurs web dans des conditions strictement contrôlées, tout en collectant des mesures relatives aux performances applicatives, à l'utilisation des cœurs CPU et à la consommation énergétique du système. Les expérimentations sont conçues de manière à être reproductibles, ce qui garantit la fiabilité et la comparabilité des résultats obtenus. Cette démarche, fondée sur l'observation, la répétition et la comparaison de différentes configurations d'affinité CPU, fournit un cadre rigoureux pour analyser l'impact du nombre de cœurs actifs, de la localité des caches et des migrations de threads sur l'efficacité énergétique. Cinq séries d'expériences ont ainsi été menées sur chaque application web, correspondant à des configurations distinctes d'affinité CPU. Cependant, le pipeline méthodologie reste le même pour toutes les expériences (voir Figure 2.1).

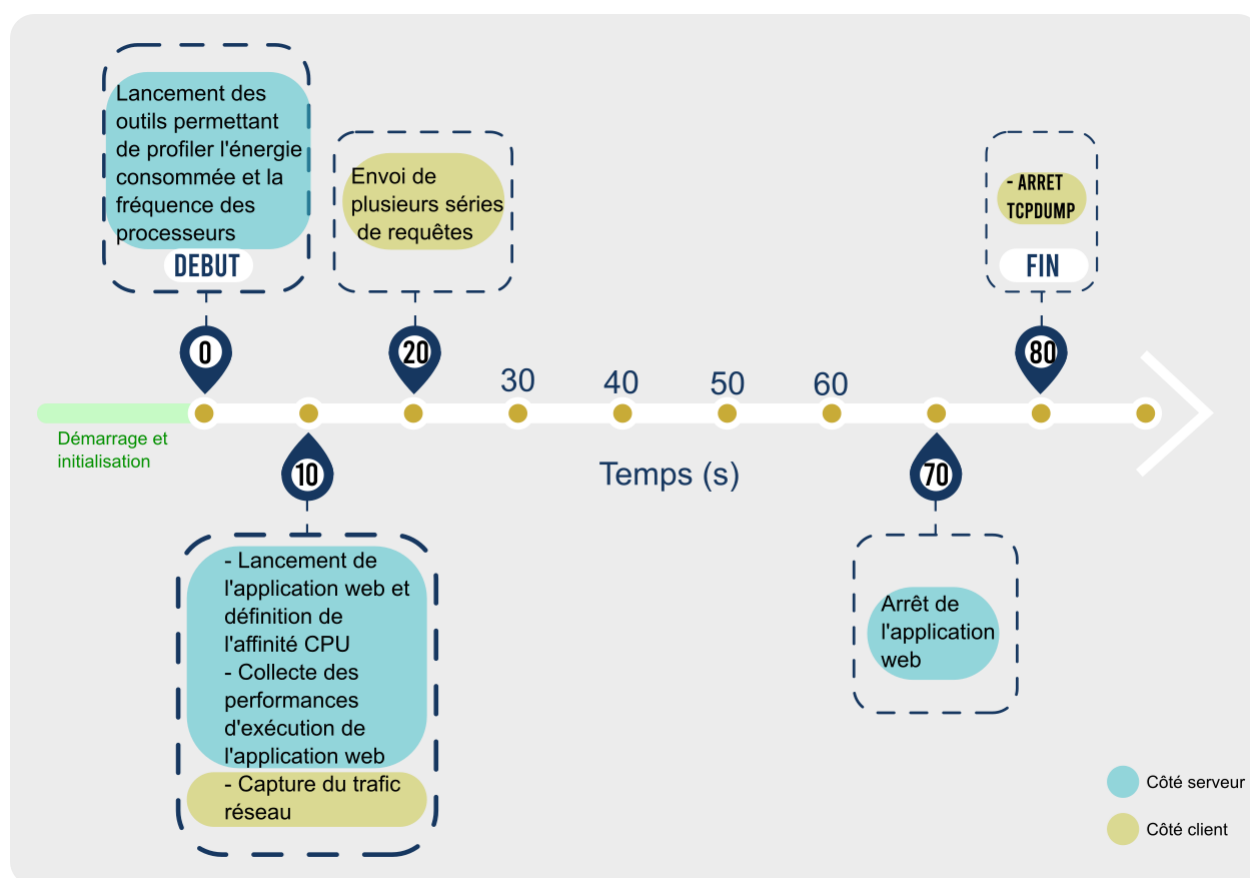


FIGURE 2.1 – Vue globale de la méthodologie expérimentale

Afin de garantir la reproductibilité des résultats, l'ensemble des scripts d'automatisation, des outils de mesure et des configurations expérimentales est mis à disposition dans un dépôt public (voir Annexe A).

2.1.2 Phase préparatoire

Cette phase préparatoire vise à garantir la reproductibilité, la comparabilité et la fiabilité des résultats expérimentaux. Elle consiste à définir précisément l'environnement matériel et logiciel, ainsi qu'à établir des scénarios expérimentaux standardisés permettant d'isoler l'impact de la configuration de l'affinité CPU sur les performances et la consommation énergétique des applications web étudiées.

2.1.2.1 Environnement matériel et logiciel

Les expériences ont été menées sur une plateforme matérielle dédiée, équipée d'un processeur multicœur disposant de 28 cœurs. Ce choix permet d'évaluer différents niveaux de parallélisme et d'observer l'impact de la restriction du nombre de cœurs sur le comportement du système.

Le système d'exploitation utilisé est Linux, choisi pour sa transparence, sa configurabilité et son support avancé des mécanismes de gestion des ressources matérielles, notamment l'affinité CPU et les interfaces de mesure énergétique. La version du noyau et les paramètres système ont été figés afin de limiter toute source de variabilité entre les expérimentations.

Les applications web étudiées reposent sur des serveurs Nginx et Node.js exécutés localement sur la machine expérimentale. La charge applicative est générée de manière contrôlée à l'aide d'un outil de simulation de requêtes Apache Benchmark, permettant de reproduire des conditions d'utilisation identiques pour chaque scénario testé.

La mesure de la consommation énergétique s'appuie sur des outils logiciels exploitant les interfaces matérielles du processeur, tels que powerstat et les compteurs RAPL. Ces outils fournissent des mesures fines de l'énergie consommée par le processeur, avec une surcharge minimale, ce qui est essentiel pour garantir la validité des résultats.

2.1.2.2 Définition des scénarios expérimentaux

Afin d'évaluer l'impact de la configuration de l'affinité CPU, plusieurs scénarios expérimentaux ont été définis pour les serveurs web choisis. Chaque scénario (S) correspond à une politique distincte d'affectation des processus applicatifs aux cœurs du processeur, comme le montre les figures 2.2 et 2.3.

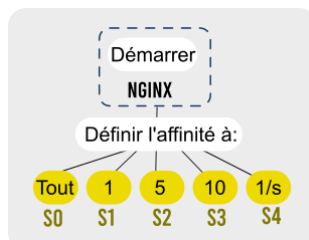


FIGURE 2.2 – Exécution de l'application web Nginx

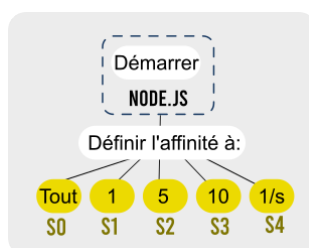


FIGURE 2.3 – Exécution de l'application web Node.js

Le scénario **S0** sert de référence et correspond au comportement par défaut du système d'exploitation, où l'ordonnanceur est libre de planifier les processus sur l'ensemble des cœurs disponibles. Les scénarios **S1** à **S3** correspondent aux scénarios de définition de l'affinité CPU respectivement à 1, 5, 10 coeurs. Enfin, le scénario **S4** introduit une dimension dynamique en

imposant une affinité CPU changeante par seconde, afin d'évaluer l'impact des migrations fréquentes de processus sur la stabilité des performances et la consommation énergétique. Chaque scénario est exécuté dans des conditions strictement identiques en termes de charge applicative et de configuration système. Les expériences sont répétées plusieurs fois afin d'évaluer la variabilité des mesures et de renforcer la robustesse statistique des résultats.

2.1.2.3 Définition des métriques observées

Les métriques retenues dans ce travail portent sur les performances applicatives, la consommation énergétique et le comportement du système lors de l'exécution des applications web. Elles sont détaillées dans le prochain tableau. Les métriques de performance, telles que la latence des requêtes et le débit, permettent d'évaluer la qualité de service perçue par l'utilisateur final. Les mesures énergétiques renseignent quant à elles sur le coût réel de l'exécution, exprimé en énergie consommée. En parallèle, des métriques issues du noyau Linux et des compteurs matériels sont collectées afin d'analyser l'impact de l'affinité CPU sur l'activité du processeur. Le temps d'utilisation CPU, la fréquence des cœurs, les migrations CPU, les changements de contexte permettent de caractériser finement le comportement du système. Ces indicateurs mettent en évidence les effets des politiques de placement des processus, notamment en termes de migrations, de localité des caches et de réveil des cœurs. Ainsi, l'ensemble de ces métriques, combinant analyse applicative, système et matérielle, constitue une base solide pour étudier l'impact de l'affinité CPU sur les performances et la consommation énergétique des applications web.

TABLE 2.1 – Métriques suivies pendant l'exécution des applications web

Métriques	Descriptions
cpu_migrations	Nombre de migrations de processus ou de threads entre cœurs.
context_switches	Nombre de changements de contexte.
cache_misses	Nombre de défauts de cache, révélant les pertes de localité et les accès mémoire plus coûteux.
cpu_utilization	Utilisation CPU globale (%), permettant d'observer la répartition de la charge selon les configurations d'affinité.

2.1.3 Protocoles expérimentaux

Les protocoles expérimentaux définissent de manière précise les étapes suivies pour la conduite des expériences, depuis l'initialisation du système jusqu'à la collecte des mesures. Leur objectif est d'assurer la comparabilité des scénarios, la réduction du bruit expérimental et la reproductibilité des résultats obtenus.

2.1.3.1 Initialisation du système

Avant chaque série d'expérimentations, le système est initialisé afin de garantir un état de départ comparable entre les différents scénarios. Cette phase comprend le redémarrage de la machine expérimentale et la vérification de l'absence de processus concurrents non essentiels. Les caches système et applicatifs sont laissés dans leur état naturel afin de reproduire un comportement réaliste, tandis que les services non requis pour l'expérimentation sont désactivés. Une période de stabilisation est respectée avant le lancement des mesures.

2.1.3.2 Déploiement de l'application et configuration de l'affinité CPU

Pour chaque scénario expérimental, l'application web étudiée (Nginx ou Node.js) est lancée avec la configuration d'affinité CPU correspondante. Les mécanismes natifs du système d'exploitation sont utilisés pour imposer les contraintes d'affinité, garantissant ainsi que l'ordonnanceur respecte les cœurs spécifiés.

Dans le cas des scénarios S0 à S3, l'affinité CPU est maintenue constante pendant toute la durée de l'expérience. Pour le scénario à affinité variable (S4), un mécanisme automatisé est mis en place afin de modifier le cœur sur lequel l'affinité CPU y est définie (le choix des cœurs se fait suivant l'ordre croissant de numérotation inscrit sur leur nom en commençant par CPU0), pour une période fixée à une seconde.

2.1.3.3 Génération de la charge applicative

La charge applicative est générée à l'aide de l'outil Apache Benchmark, configuré de manière identique pour l'ensemble des scénarios.

- Pour l'application Nginx, un nombre total de 50.000 requêtes subdivisé en 5 séries de 10.000 sans concurrence est envoyé, par intervalle de temps variable ; afin de simuler un cas réel.
- Pour l'application Node.js, un nombre total de 60 requêtes sans concurrence est envoyé en une série.

La génération de charge est déclenchée uniquement après la mise en place complète de la configuration d'affinité CPU, afin de s'assurer que les mesures reflètent exclusivement l'impact du scénario testé.

2.1.3.4 Collecte des mesures

La collecte des données est réalisée de manière synchronisée avec l'exécution de la charge applicative.

Au niveau de la **machine Serveur**, les performances CPU sont mesurées à l'aide de l'outil **perf stat**, permettant de collecter des métriques tels que les migrations CPU, les changements

de contexte, le temps d'utilisation du CPU. La consommation énergétique est mesurée à l'aide de **powerstat**, exploitant les interfaces RAPL du processeur pour estimer l'énergie consommée pendant chaque expérimentation.

Au niveau de la **machine Client**, le trafic réseau est capturé à l'aide de **tcpdump**, afin d'analyser les performances (Temps de latence et débit) de l'application web. Mais également afin de vérifier la cohérence de la charge générée et d'identifier d'éventuelles anomalies.

Toutes les mesures sont enregistrées sous forme de fichiers horodatés, facilitant leur analyse ultérieure et garantissant la traçabilité des résultats.

2.1.4 Méthodes d'analyse

Les méthodes d'analyse ont pour objectif de transformer les mesures brutes collectées lors des expérimentations en indicateurs exploitables permettant d'évaluer l'impact de la configuration de l'affinité CPU sur les performances et la consommation énergétique des applications web étudiées. Cette phase vise à garantir une interprétation rigoureuse, objective et comparable des résultats obtenus pour l'ensemble des scénarios expérimentaux.

2.1.4.1 Prétraitement des données

Les données issues des différents outils de mesure sont d'abord prétraitées afin d'assurer leur cohérence et leur exploitabilité. Cette étape comprend la synchronisation temporelle des mesures de performance, de consommation énergétique et de trafic réseau, ainsi que l'élimination des valeurs aberrantes identifiées lors de la phase de validation expérimentale.

Les résultats de chaque exécution sont agrégés sous forme de séries statistiques, en conservant les informations nécessaires à l'analyse de la variabilité, telles que les valeurs moyennes, les écarts-types et les intervalles de confiance.

2.1.4.2 Analyse des performances

L'analyse des performances repose principalement sur l'étude du temps de réponse, du débit et des compteurs matériels collectés via perf stat. Ces indicateurs permettent d'évaluer l'efficacité de l'ordonnancement et l'impact de la restriction du nombre de cœurs sur le traitement des requêtes.

Une comparaison inter-scénarios est réalisée afin d'identifier les configurations d'affinité CPU offrant le meilleur compromis entre performance et utilisation des ressources. Une attention particulière est portée aux effets de saturation, aux changements de contexte et aux migrations de processus, qui peuvent dégrader la localité des caches et impacter les performances globales.

2.1.4.3 Analyse de la consommation énergétique

La consommation énergétique est analysée à partir des mesures fournies par Powerstat et des compteurs RAPL. Les données sont exploitées pour estimer l'énergie instantanée consommée durant chaque scénario expérimental.

L'analyse compare les scénarios afin d'identifier les configurations permettant de réduire l'énergie consommée tout en maintenant un niveau de performance acceptable. Cette approche met en évidence l'existence de compromis entre performance maximale et efficacité énergétique, en particulier lorsque l'affinité CPU est définie à un nombre de cœurs réduit.

2.1.4.4 Analyse combinée performance-énergie

Une analyse conjointe des performances et de la consommation énergétique est réalisée afin d'évaluer l'efficacité globale du système. Cette approche permet d'identifier les configurations d'affinité CPU offrant le meilleur compromis entre rapidité de traitement et efficacité énergétique, en mettant en évidence les situations où une légère dégradation des performances peut conduire à une réduction significative de la consommation énergétique.

2.1.4.5 Comparaison entre technologies applicatives

Les résultats obtenus pour les différentes applications web étudiées sont comparés afin d'analyser l'influence de l'architecture logicielle sur l'impact de l'affinité CPU. Cette comparaison met en évidence les différences de sensibilité aux restrictions de cœurs et aux migrations de processus, en fonction du modèle d'exécution propre à chaque technologie.

2.1.4.6 Validation et interprétation des résultats

Enfin, les résultats sont interprétés à la lumière des hypothèses de recherche formulées initialement. Les tendances observées sont discutées en tenant compte des limites expérimentales et des hypothèses sous-jacentes. Cette phase permet de valider ou de nuancer les conclusions tirées des expérimentations et de dégager des enseignements généraux sur le rôle de l'affinité CPU dans l'optimisation des performances et de la consommation énergétique des applications web.

Les méthodes d'analyse décrites ci-dessus permettent d'interpréter rigoureusement les données expérimentales. La section suivante présente la modélisation du cadre d'analyse et les éléments théoriques soutenant les choix expérimentaux.

2.1.5 Modélisation du cadre d'analyse

Cette section présente la modélisation conceptuelle et analytique du cadre d'analyse utilisé pour interpréter les résultats expérimentaux. L'objectif de cette modélisation est de formaliser les relations entre la configuration de l'affinité CPU, le comportement du système d'exploitation et



les indicateurs de performance et de consommation énergétique observés. Elle fournit un support théorique permettant de structurer l'analyse et de justifier les choix expérimentaux effectués.

2.1.5.1 Modèle conceptuel du système étudié

Le système étudié est modélisé comme un ensemble de composants interagissant selon une chaîne de traitement bien définie : charge applicative -> application web -> système d'exploitation -> matériel.

La charge applicative génère un flux de requêtes HTTP, caractérisé par un taux d'arrivée et sans concurrence. L'application web (Nginx ou Node.js) traite ces requêtes à l'aide de processus et de threads gérés par le système d'exploitation. Le noyau Linux assure l'ordonnancement de ces entités d'exécution sur les cœurs CPU disponibles, conformément à la configuration de l'affinité CPU imposée. Enfin, le matériel exécute les instructions, consomme de l'énergie et fournit des compteurs de performance exploitables.

2.1.5.2 Modélisation de l'affinité CPU et des performances

L'affinité CPU est modélisée comme une contrainte de planification limitant l'ensemble des cœurs sur lesquels un processus ou un groupe de threads peut être exécuté. Les performances du système sont modélisées à travers des indicateurs observables tels que le temps de réponse moyen et le débit(requêtes par seconde). Ces indicateurs dépendent du nombre de cœurs disponibles, de la charge appliquée et de l'efficacité de l'ordonnancement.

2.1.5.3 Modélisation de la consommation énergétique

La consommation énergétique est modélisée à partir de la puissance moyenne. Hors, la puissance consommée dépend du nombre de cœurs actifs, de leur fréquence et du niveau d'activité induit par la charge applicative. En définissant l'affinité CPU à un nombre de cœurs plus bas, il est possible de réduire le nombre de cœurs actifs et d'améliorer l'efficacité énergétique globale du système.

2.1.5.4 Utilisation de la modélisation dans l'analyse

La modélisation proposée ne vise pas à prédire précisément le comportement du système, mais à fournir un cadre analytique cohérent pour interpréter les résultats expérimentaux. Elle permet de formuler des hypothèses sur l'impact de la restriction des cœurs et de guider l'analyse comparative des scénarios.

Les résultats expérimentaux sont ainsi analysés à la lumière de ce cadre, en mettant en évidence les configurations d'affinité CPU offrant le meilleur compromis entre performance et consommation énergétique.

La modélisation du cadre d'analyse fournit les bases théoriques nécessaires à l'interprétation des résultats. La partie suivante est consacré aux fondements méthodologiques de la recherche.

2.2 FONDEMENTS MÉTHODOLOGIQUES DE LA RECHERCHE

Cette section explicite les fondements scientifiques et méthodologiques qui sous-tendent l'approche adoptée dans ce travail. Elle vise à justifier le choix de la démarche expérimentale retenue et à positionner l'étude dans un cadre scientifique reconnu en évaluation des performances et de l'efficacité énergétique des systèmes informatiques.

2.2.1 Justification du choix de l'approche

Le choix d'une approche expérimentale et comparative s'impose dans le cadre de cette étude en raison de la nature du phénomène analysé. L'impact de la configuration de l'affinité CPU sur les performances et la consommation énergétique résulte d'interactions complexes entre le matériel, le système d'exploitation et les applications. Ces interactions ne peuvent être entièrement capturées par des modèles analytiques ou des simulations abstraites, qui reposent souvent sur des hypothèses simplificatrices.

L'expérimentation contrôlée permet d'observer directement le comportement réel du système dans des conditions maîtrisées. En définissant des scénarios expérimentaux précis et reproductibles, il devient possible d'isoler l'effet de la variable étudiée, tout en maintenant constantes les autres composantes du système. La comparaison systématique entre plusieurs configurations permet ainsi d'identifier des tendances robustes et de quantifier les compromis entre performance et consommation énergétique.

Par ailleurs, l'approche comparative retenue, fondée sur l'étude de plusieurs applications web représentatives et de plusieurs configurations d'affinité CPU, permet de généraliser les observations au-delà d'un cas particulier. Elle offre une vision plus large de l'influence des décisions du système d'exploitation sur des architectures logicielles distinctes, renforçant ainsi la portée des résultats.

2.2.2 Cadre scientifique de la recherche

La méthodologie adoptée s'inscrit dans le champ de la recherche en systèmes informatiques, et plus précisément dans les domaines de l'évaluation des performances, de l'ordonnancement des processus et de l'efficacité énergétique des architectures multicœurs. De nombreux travaux ont montré que les politiques de planification et la gestion de la localité des données jouent un rôle déterminant dans les performances des applications parallèles et des serveurs.

Ce travail s'appuie sur les concepts fondamentaux de l'ordonnancement multicœur, de la localité des caches et de la gestion dynamique de l'énergie, tout en mettant en évidence le rôle



central du système d'exploitation comme intermédiaire entre le matériel et les applications. L'étude de l'affinité CPU s'inscrit dans cette perspective, en considérant cette dernière comme un levier logiciel permettant d'influencer à la fois les performances et la consommation énergétique.

En outre, le choix d'applications web comme cas d'étude s'aligne avec les pratiques courantes de la littérature scientifique et industrielle, où ces applications sont fréquemment utilisées pour évaluer le comportement des systèmes en conditions réalistes. L'intégration de mesures matérielles, telles que les compteurs de performance et les interfaces énergétiques, permet enfin d'ancrer l'analyse dans un cadre expérimental rigoureux et quantitativement exploitable.

2.3 BILAN

Ce chapitre a présenté la méthodologie de recherche adoptée pour analyser l'impact de la configuration de l'affinité CPU sur les performances et la consommation énergétique des applications web exécutées sous Linux. L'approche retenue repose sur une expérimentation contrôlée et comparative, permettant d'observer de manière rigoureuse les interactions entre le système d'exploitation, le matériel et les applications.

La phase préparatoire a permis de définir un environnement expérimental stable et reproductible, en précisant les configurations matérielles et logicielles, les outils de mesure et les scénarios expérimentaux. Cette étape a été essentielle pour limiter les sources de variabilité et garantir la comparabilité des résultats entre les différents scénarios d'affinité CPU.

Les protocoles expérimentaux ont ensuite détaillé les étapes de déploiement des applications, de configuration des paramètres système, de génération de la charge et de collecte des mesures. L'utilisation d'outils de profilage adaptés et peu intrusifs a permis de collecter des données fiables sur les performances, la consommation énergétique et le comportement réseau, tout en minimisant l'impact des mesures sur le système étudié.

Les méthodes d'analyse ont fourni un cadre rigoureux pour exploiter les données collectées, en combinant des indicateurs de performance et d'efficacité énergétique. L'analyse comparative des scénarios et des technologies applicatives étudiées permet de mettre en évidence les compromis entre performance et consommation énergétique, ainsi que l'influence de l'architecture logicielle sur l'impact de l'affinité CPU.

Enfin, les fondements méthodologiques ont permis de situer cette étude dans un cadre scientifique plus large, en justifiant le choix de l'approche expérimentale et en soulignant sa pertinence au regard des travaux existants en évaluation des systèmes informatiques. Ce cadre méthodologique constitue une base solide pour l'interprétation des résultats expérimentaux présentés dans le chapitre suivant.

Le chapitre suivant est consacré à la présentation et à l'analyse des résultats expérimentaux obtenus à partir de la méthodologie décrite ci-dessus.

IMPLÉMENTATION ET RÉSULTATS

C *e chapitre détaille les outils et méthodes utilisés pour mettre en œuvre notre solution. Ensuite il présente les résultats obtenus, en soulignant les performances et observations principales. Enfin, il compare notre approche aux solutions existantes pour évaluer sa pertinence, son efficacité et sa contribution au domaine étudié.*



3.1 IMPLÉMENTATION DU PROTOCOLE EXPÉRIMENTAL

Cette section décrit l'infrastructure matérielle et les outils logiciels utilisés pour mener les expérimentations. Leur choix a été guidé par la nécessité d'assurer précision, reproductibilité et contrôle fin des conditions expérimentales.

3.1.1 Ressources matérielles

Les expériences ont été menées sur l'infrastructure CloudLab, offrant un environnement contrôlé et reproductible. Chaque poste de test est minutieusement configuré.

3.1.1.1 Infrastructure CloudLab

CloudLab est une plateforme pour tester des systèmes informatiques et le cloud. Elle permet aux chercheurs d'utiliser des machines et du matériel dédiés, sans interférence d'autres expériences, ce qui rend les résultats fiables. On peut y créer et partager des environnements complets que d'autres chercheurs peuvent reproduire pour vérifier ou continuer les expériences [19]. Le fonctionnement de la plateforme repose sur la notion de projets et profils :

- Un projet correspond à un regroupement d'utilisateurs (enseignants-chercheurs, doctorants, étudiants, ingénieurs) travaillant sur un objectif scientifique commun.
- Un profil décrit l'intégralité d'un environnement expérimental, incluant les ressources matérielles (nœuds physiques, machines virtuelles, topologie réseau) et logicielles (systèmes d'exploitation, packages, images disque). Chaque profil peut être instancié pour créer une expérience indépendante, isolée et parfaitement contrôlée.

Techniquement, CloudLab donne accès à plusieurs groupes de machines puissantes. Les utilisateurs ont un contrôle total (accès root) pour installer et configurer leurs logiciels comme ils veulent. Les machines sont temporaires, mais on peut sauvegarder leur état ou les réutiliser. CloudLab sert aux chercheurs comme aux enseignants : les étudiants peuvent y apprendre en expérimentant sur des systèmes isolés et reproductibles, et les chercheurs peuvent tester la performance et la fiabilité de systèmes complexes [19].

3.1.1.2 Spécifications techniques des postes expérimentaux

Les expérimentations ont été conduites sur des machines physiques strictement identiques, dédiées exclusivement à cette étude, afin de garantir des conditions d'exécution stables, contrôlées et reproductibles. L'utilisation de machines non virtualisées permet d'éliminer les biais liés à la couche d'abstraction et d'observer directement l'impact des décisions du système d'exploitation sur le matériel.



Les processeurs multicœurs offrent un cadre pertinent pour analyser l'effet de la configuration de l'affinité CPU, notamment en ce qui concerne le nombre de cœurs actifs, la localité des caches et les migrations de threads. La capacité de la mémoire vive a été dimensionnée de manière à éviter toute contention mémoire susceptible d'influencer les résultats, tandis que l'utilisation d'un stockage rapide permet de minimiser l'impact des opérations d'entrées/sorties sur les mesures.

Ce choix d'environnement expérimental vise à s'assurer que les variations observées en termes de performances applicatives et de consommation énergétique sont principalement imputables aux paramètres étudiés, en particulier à la configuration de l'affinité CPU, et non à des facteurs externes ou matériels non maîtrisés.

Les caractéristiques matérielles et logicielles des systèmes utilisés sont présentées dans le tableau 3.1 :

TABLE 3.1 – Spécifications matérielles des machines utilisées

Composants	Spécifications
Général	— Type : c6320 — Année : 2018 — Cluster : CloudLab Clemson
CPU Intel E5-2683v3 6142 @ 2.00 GHz	— Nombre total de threads : 56 — Vitesse de base (GHz) : 2 — Processeurs physiques : 2 — Cœurs par processeur : 14 — Threads par cœur : 2
RAM	— DRam (Go) : 384 Go — Swap (Go) : 3
Système	— Système d'exploitation : Ubuntu Linux 24.04 LTS (noyau Linux 6.8.0-71-generic) — Architecture : x86_64 (64 bits)

3.1.2 Environnements logiciels

Les expérimentations ont été réalisées en utilisant un ensemble d'outils logiciels permettant l'implémentation d'applications web, la génération des requêtes, le profilage des performances et la surveillance du système.



3.1.2.1 Système d'exploitation : Linux

Le système d'exploitation **Linux** a été retenu comme système d'exploitation pour cette étude en raison de sa large adoption, de sa transparence et de la richesse de ses mécanismes de gestion des ressources, en particulier dans les environnements serveurs. Linux constitue aujourd'hui la plateforme dominante pour l'hébergement d'applications web et de services cloud [20], ce qui confère aux résultats obtenus une forte pertinence pratique et industrielle.

De plus, Linux offre un contrôle fin et explicite des mécanismes d'ordonnancement et de gestion de l'énergie, notamment à travers la gestion de l'affinité CPU, les politiques de planification des processus et des threads, ainsi que les mécanismes de gestion dynamique de la fréquence et des états d'inactivité des cœurs. Ces fonctionnalités sont directement accessibles via des interfaces documentées, permettant de configurer précisément les paramètres étudiés et d'en analyser l'impact de manière reproductible. À l'inverse, d'autres systèmes d'exploitation, tels que **Windows** ou **macOS**, proposent des abstractions plus opaques et un accès plus restreint aux mécanismes internes du noyau, limitant la capacité à isoler et analyser finement l'effet de décisions de planification spécifiques.

Enfin, en plus d'une documentation abondante, Linux dispose d'un écosystème mature d'outils de mesure et d'analyse des performances et de la consommation énergétique, tels que `perf`, `powertop`, `turbostat` ou les interfaces `RAPL`. Ces outils permettent d'observer directement l'impact des configurations expérimentales sur le comportement du système, sans introduire de surcharge significative. Ce niveau d'instrumentation, difficilement atteignable de manière équivalente sur d'autres systèmes d'exploitation, renforce la pertinence du choix de Linux pour une étude centrée sur les interactions entre logiciel, matériel et efficacité énergétique.

3.1.2.2 Applications web

Les applications web constituent aujourd'hui l'une des charges de travail les plus répandues dans les environnements serveurs, notamment au sein des infrastructures cloud et des centres de données. Leur architecture client–serveur se caractérise par un grand nombre de requêtes concurrentes, des traitements généralement courts et une forte dépendance aux mécanismes d'entrées/sorties réseau.

Pendant leur fonctionnement, les applications web sollicitent intensivement l'ordonnanceur du système d'exploitation. À charge applicative identique, des politiques différentes d'ordonnancement peuvent entraîner des variations significatives du temps de réponse, du débit et de l'énergie consommée. Dans le cadre de cette étude, deux applications web, présentant des profils de charge différents, ont été utilisées afin d'évaluer l'influence de l'affinité CPU selon la nature des traitements effectués.

- **Nginx** : est un serveur web HTTP, un proxy inverse, un cache de contenu, un système d'équilibrage de charge, un serveur proxy TCP/UDP et un serveur proxy de messagerie.

Initialement développé par Igor Sysoev, il est distribué sous licence BSD. Reconnu pour sa



flexibilité, ses hautes performances et sa faible consommation de ressources, nginx est : le serveur web le plus populaire au monde [21], une des images Docker les plus utilisées [22], et alimente de nombreux contrôleurs d'entrée pour Kubernetes [23]. Il repose sur une architecture événementielle non bloquante, dans laquelle un nombre limité de processus workers assure le traitement d'un grand volume de requêtes simultanées.

Cette étude a été réalisée avec l'application par défaut du serveur Nginx, distribuant du contenu statique. Cette application ne réalise aucun traitement computationnel significatif côté serveur et repose principalement sur des opérations d'entrées/sorties, telles que la réception des requêtes, la lecture des fichiers et l'envoi des réponses au client. Le diagramme de séquence correspondant, présenté à la Figure 3.1, illustre ce cheminement.

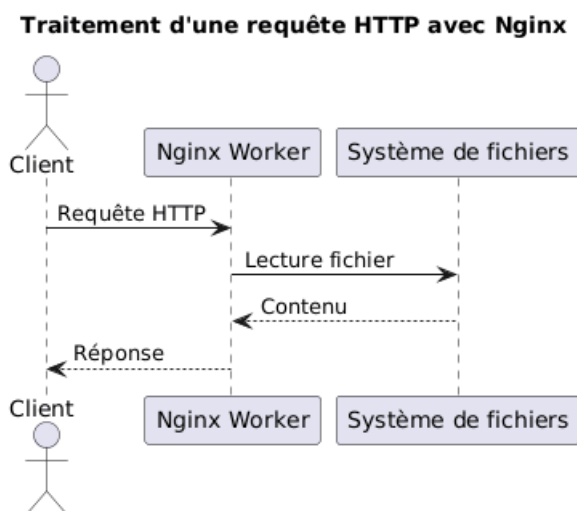


FIGURE 3.1 – Diagramme de séquences de l'application Nginx

Afin de s'établir sur le choix de ce serveur web, le tableau suivant présente un résumé comparatif aux autres serveurs concurrents.

TABLE 3.2 – Tableau comparatif des serveurs web

Caractéristiques	Node.js	Nginx	Apache
Utilisation principale	Développement d'applications dynamiques en temps réel	Service de fichiers statiques haute performance, proxy inverse, répartition de charge	Service web généraliste, contenu dynamique avec modules
Architecture	Monothread, événementielle, E/S non bloquantes	Événementielle, asynchrone	Multiprocessus/multithread, E/S bloquantes
Performances	Excellentes pour les applications temps réel à forte intensité d'E/S, mais pas pour les tâches gourmandes en ressources CPU	Très rapides pour le contenu statique et une forte concurrence	Plus lents que Nginx pour les fichiers statiques en cas de forte charge, mais très flexibles
Configuration	Configuration par programmation au sein de la logique applicative	Configuration via des fichiers de configuration simples et efficaces	Configuration via des fichiers de configuration

Comparé à des serveurs web comme Apache HTTP Server et Node.js, Nginx se distingue par une faible empreinte mémoire et une excellente scalabilité sous charge élevée. Ces caractéristiques en font un support adapté pour analyser l'impact de l'affinité CPU dans un contexte dominé par les entrées/sorties.

- **Node.js** : Node.js est un environnement d'exécution JavaScript open source et multiplateforme. Il exécute le moteur JavaScript V8, le cœur de Google Chrome, en dehors du navigateur. Cela lui confère d'excellentes performances. Une application Node.js s'exécute dans un seul processus, sans créer un nouveau thread pour chaque requête. Elle fournit un ensemble de primitives d'E/S asynchrones dans sa bibliothèque standard, empêchant ainsi le blocage du code JavaScript. De plus, les bibliothèques Node.js sont généralement écrites selon des paradigmes non bloquants [24]. Lorsqu'une opération d'E/S est effectuée par Node.js, comme la lecture sur le réseau, l'accès à une base de données ou au système de fichiers, au lieu de bloquer le thread et de gaspiller des ressources CPU, Node.js reprend l'opération dès réception de la réponse. Cela permet à Node.js de gérer des milliers de connexions simultanées avec un seul serveur sans la complexité de la gestion de la concurrence des threads, source potentielle de bogues.

Node.js présente un avantage unique : des millions de développeurs front-end qui écrivent du JavaScript pour le navigateur peuvent désormais écrire le code côté serveur en plus du code côté client sans avoir à apprendre un langage complètement différent [24]. Dans le

cadre de cette étude, une application spécifique réalisant une opération computationnelle intensive à chaque requête a été développée en Node.js. Cette application exécute un grand nombre d'itérations et de calculs arithmétiques, générant une charge fortement dépendante du processeur. Le diagramme de séquence associé, présenté à la Figure 3.2, met en évidence la phase centrale de calcul bloquant qui domine le temps de traitement.

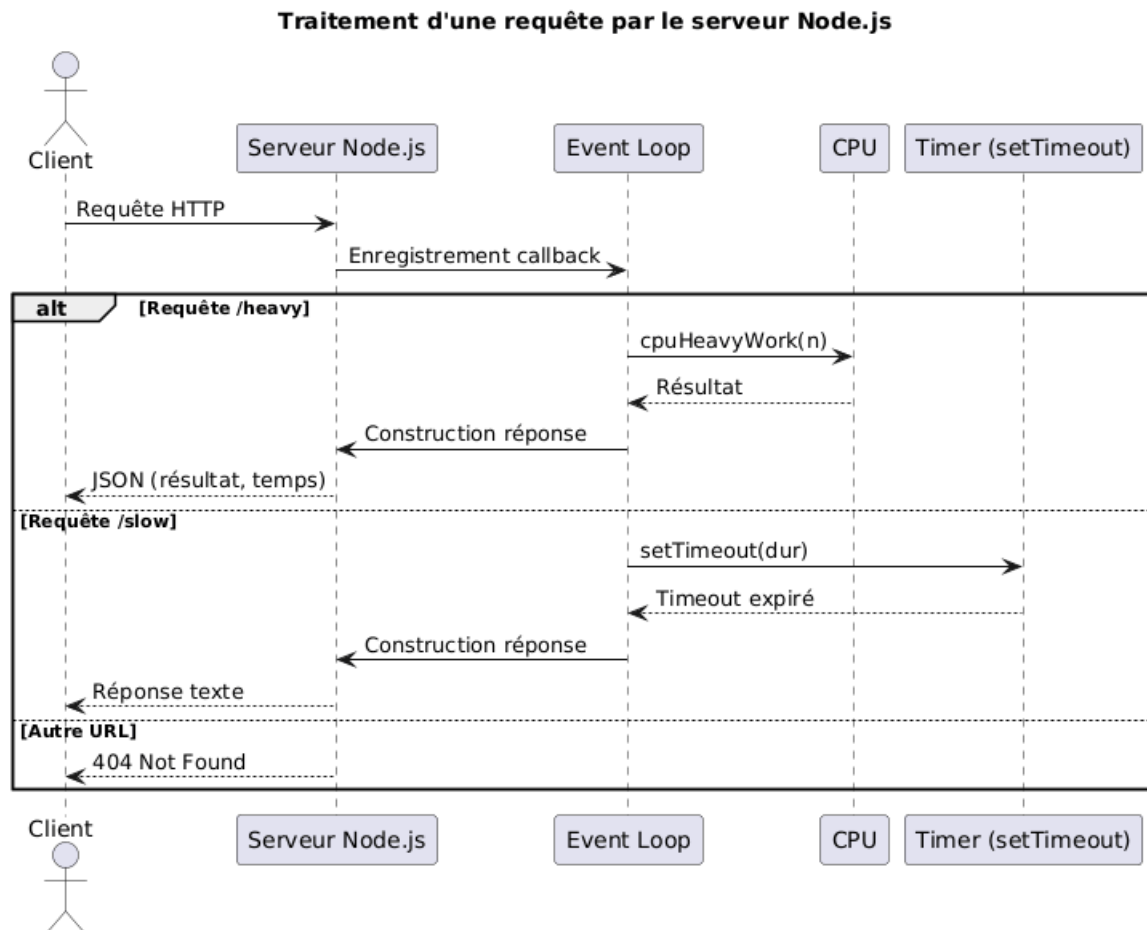


FIGURE 3.2 – Diagramme de séquences de l'application Nginx

Au vue du tableau 3.2, Node.js a été retenu comme serveur applicatif dans cette étude en raison de son modèle d'exécution événementiel principalement monothreadé, reposant sur une boucle d'événements. Cette architecture rend les applications particulièrement sensibles aux charges CPU intensives, susceptibles d'affecter directement les performances perçues. Ce choix permet ainsi d'évaluer l'impact des mécanismes d'ordonnancement et d'affinité CPU dans un contexte contraignant. Par ailleurs, Node.js est largement utilisé dans les architectures modernes de services web, ce qui confère aux résultats obtenus une pertinence pratique. Enfin, sa facilité d'instrumentation et de déploiement favorise la reproductibilité des expérimentations.

3.1.2.3 Génération des requêtes

Les requêtes HTTP utilisées dans le cadre de cette étude ont été simulées à l'aide de l'outil **Apache Benchmark (ab)**. Cet outil permet de générer une charge contrôlée et reproductible sur un serveur web en envoyant un nombre configurable de requêtes HTTP concurrentes ou pas, vers une cible donnée. Il fournit des métriques essentielles telles que le temps de réponse, le débit et la distribution des latences, qui sont directement exploitables pour l'analyse des performances applicatives. Afin de justifier le choix d'Apache Benchmark par rapport à d'autres outils de génération de charge couramment utilisés, une comparaison synthétique avec les outils wrk et JMeter est présentée dans le Tableau 3.3.

TABLE 3.3 – Tableau comparatif des outils de génération des requêtes

Composants	Apache Benchmark	wrk	JMeter
Surcharge côté client	Faible	Très faible	Élevée
Modélisation de scénari	Non	Limitée	Avancée
Charge maximale générable	Modérée	Très élevée	Élevée

Bien que des outils tels que wrk offrent une capacité de génération de charge plus élevée, ils sont principalement conçus pour pousser les serveurs à leurs limites et nécessitent parfois des scripts supplémentaires, ce qui peut introduire une variabilité non souhaitée. À l'inverse, JMeter, bien qu'extrêmement flexible, est davantage orienté vers la modélisation de scénarios applicatifs complexes et introduit une surcharge logicielle significative, peu compatible avec une étude centrée sur les mécanismes du système d'exploitation.

Ainsi, Apache Benchmark représente un compromis pertinent entre simplicité, reproductibilité et précision des mesures, ce qui en fait un outil particulièrement adapté à l'évaluation de l'impact de l'affinité CPU dans un contexte expérimental contrôlé.

3.1.2.4 Outils de profilage

Une analyse comparative a été menée entre plusieurs outils de profilage disponibles sous Linux. Les outils ont été évalués selon leur capacité à mesurer les dimensions clés étudiées (performances CPU, consommation énergétique et trafic réseau) tout en garantissant une surcharge minimale et une bonne reproductibilité des résultats. Les tableaux suivants présentent le résumé de cette analyse :

TABLE 3.4 – Tableau comparatif des outils de profilage CPU et performances

Outil	Perf stat	Top/Htop	Vmstat
Métriques mesurées	Cycles, instructions, cache, context switches	Utilisation CPU	Statistiques système
Niveau d'intrusion	Faible	Très faible	Très faible

Comme le montre le tableau 3.4, Perf stat permet de collecter des compteurs matériels précis directement exposés par le processeur, avec une surcharge minimale, ce qui le rend particulièrement adapté à l'étude de l'impact de l'affinité CPU sur les performances.

TABLE 3.5 – Tableau comparatif des outils de mesure de la consommation énergétique

Outil	Powerstat	Powertop	Outils externes
Source des données	RAPL	Estimations dynamiques	Capteurs matériels
Précision	Élevée	Moyenne	Très Élevée
Disponibilité	Très Élevée	Très Élevée	Faible

Comme le montre le tableau 3.5, Powerstat fournit des mesures d'énergie élevée sur des intervalles contrôlés, adaptées aux expériences comparatives et répétables, sans nécessiter de matériel externe.

TABLE 3.6 – Tableau comparatif des outils d'analyse du trafic réseau

Outil	Tcpdump	Wireshark	iftop
Type d'analyse	Capture de paquets	Analyse graphique	Débit instantané
Niveau de détails	Très Élevée	Très Élevée	Faible
Surcharge	Très faible	Faible	Faible

Comme le montre le tableau 3.13, Tcpdump permet une capture fine et scriptable du trafic réseau, indispensable pour vérifier la charge générée sans introduire une surcharge significative sur le serveur.

3.2 RÉSULTATS EXPÉRIMENTAUX

Cette section présente les résultats expérimentaux obtenus afin d'évaluer l'impact de la configuration de l'affinité CPU sur les performances et la consommation énergétique des deux serveurs web choisis : le serveur Nginx et le serveur NodeJs.

3.2.1 Résultats pour le serveur web Nginx

Les expériences menées avec le serveur Nginx reposent sur une application servant du contenu statique dans un contexte dominé par les entrées/sorties, avec l’envoi de cinq séries de dix mille requêtes.

3.2.1.1 Performances applicatives

— Latence

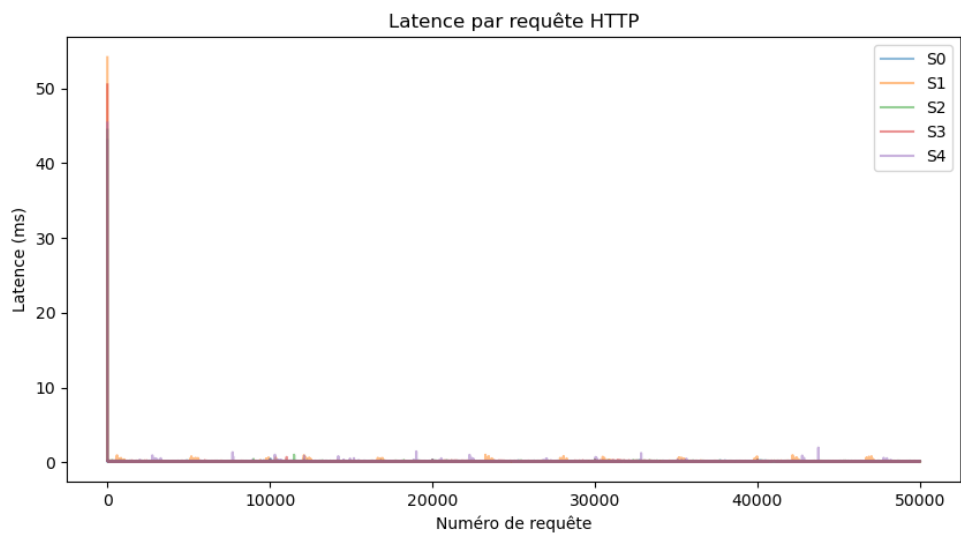


FIGURE 3.3 – Latence des requêtes par affinité CPU - Nginx

Avec pour résultats statistiques :

TABLE 3.7 – Tableau comparatif des résultats statistiques des latences

Scénario	S0	S1	S2	S3	S4
Moyenne	0,11123768	0,12384164	0,12909876	0,13474648	0,11376884
Médiane	0,104	0,104	0,140	0,140	0,097
99th percentile	0,181	0,192	0,188	0,194	0,184

Selon l’article *Analysing the Latency of WWW Applications* : La courbe la plus performante est celle qui minimise la latence de queue (Percentile 99) sans sacrifier excessivement la médiane [25]. Ainsi, nous observons les caractéristiques suivantes :

- * **S0** : Meilleur 99ème Percentile, bonne médiane, meilleure moyenne.
- * **S4** : Meilleure médiane, moyenne correcte, Percentile 99 acceptable.
- * **S2** : Médiane élevée, 99ème Percentile correct.
- * **S1** : Bonne médiane mais 99ème Percentile dégradé.
- * **S3** : mauvais sur tous les axes.



Observation

Le scénario le plus performant est S0 (Affinité sur tous les cœurs) car il minimise la latence de queue (99ème Percentile), conserve une bonne médiane, présente la distribution la plus saine.

— Débit

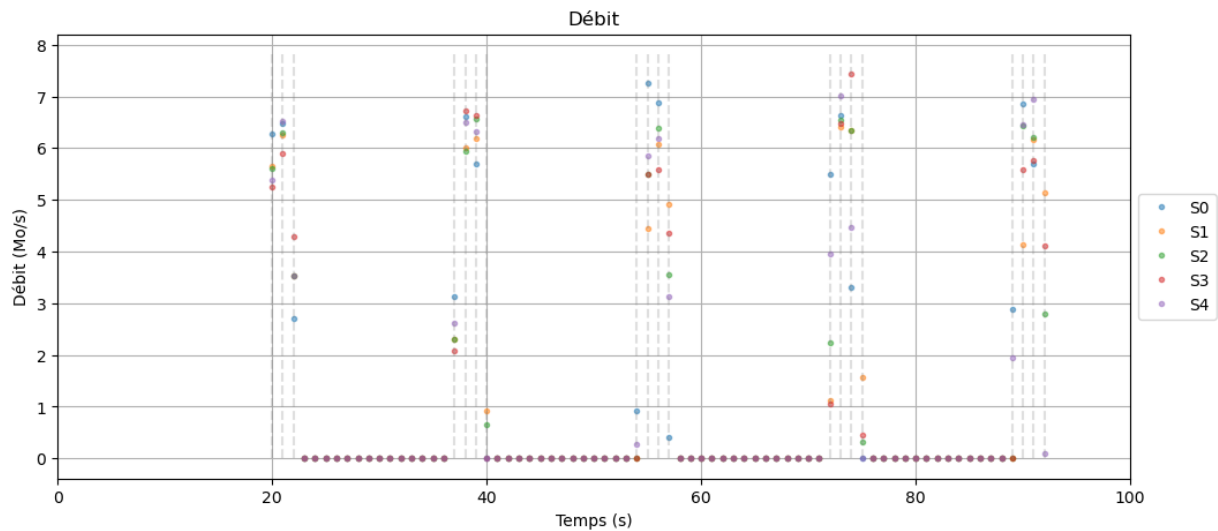


FIGURE 3.4 – Débit des requêtes par affinité CPU - Nginx

Avec pour résultats statistiques :

TABLE 3.8 – Tableau comparatif des résultats statistiques des débits

Scénario	S0	S1	S2	S3	S4
Moyenne	4,824	4,541	4,541	4,541	4,541
Médiane	5,702	5,138	5,608	5,493	5,393
99th percentile	7,197	6,395	6,560	7,325	7

Observation

Ces résultats montrent des débits plus élevés des scénarios S0 et S4 lors du traitement des requêtes.

3.2.1.2 Consommation énergétique

— **Fréquence** Les figures suivantes représentent les fréquences de tous les cœurs du processeur afin d’observer l’effet de la politique d’ordonnancement des processus, de détecter les réveils de cœurs et d’identifier les conséquences des migrations de processus.

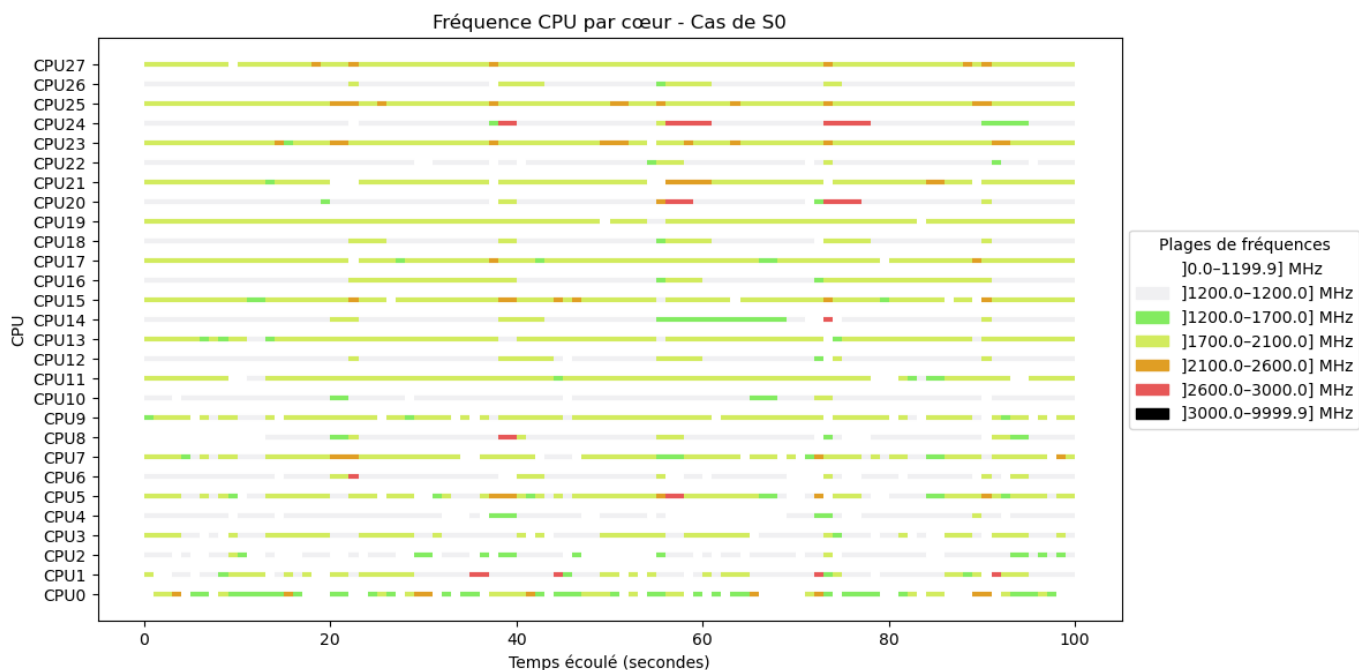


FIGURE 3.5 – Fréquence des cœurs du processeur avec pour affinité tous les cœurs - Nginx

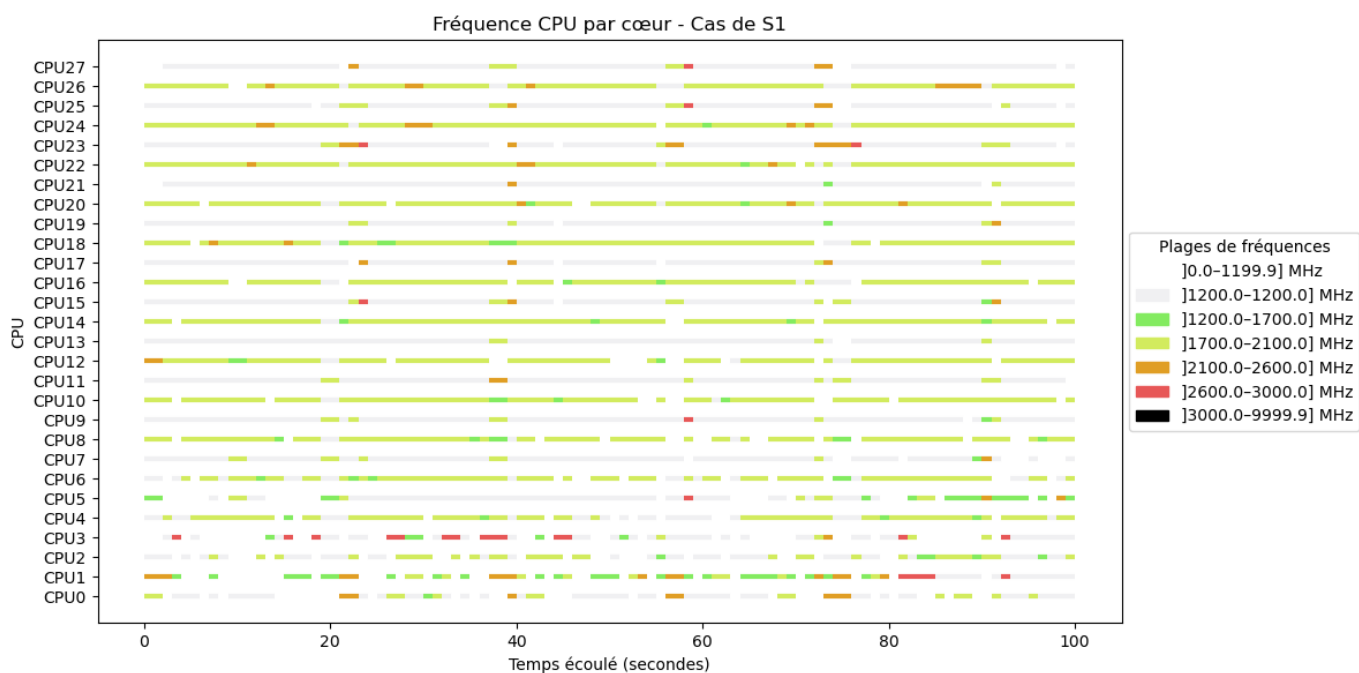


FIGURE 3.6 – Fréquence des cœurs du processeur avec pour affinité 1 cœur (CPU0) - Nginx

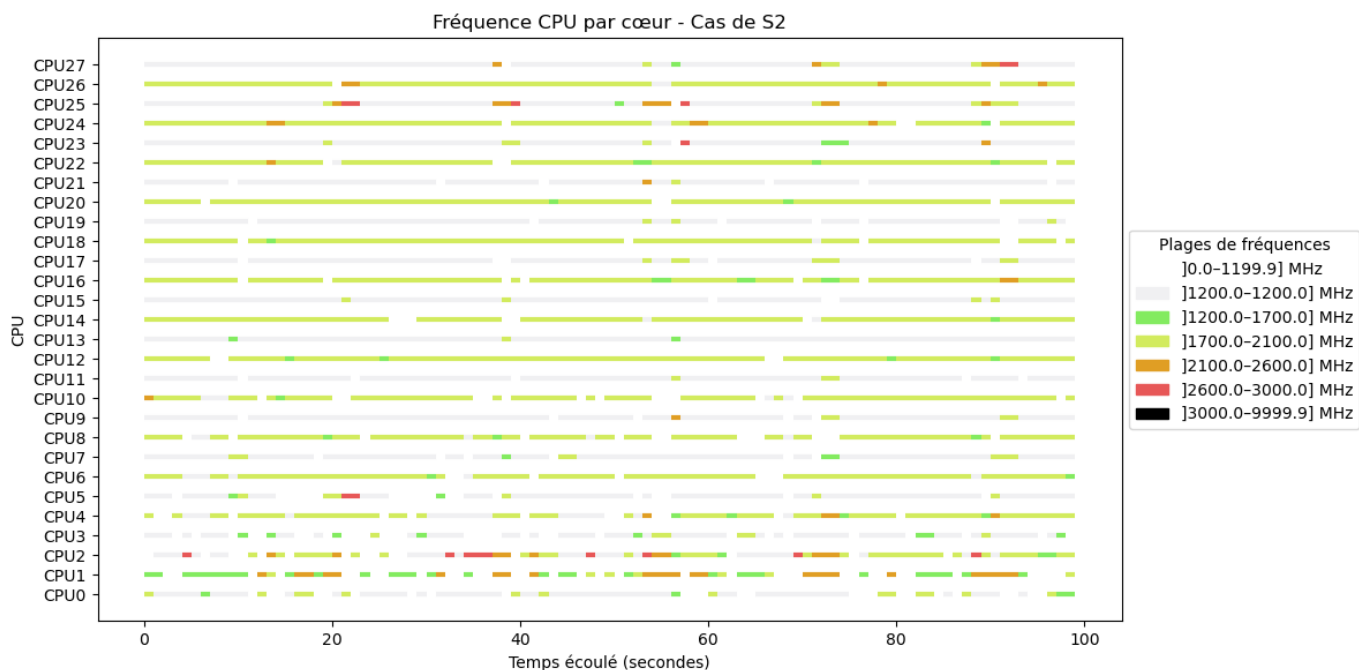


FIGURE 3.7 – Fréquence des cœurs du processeur avec pour affinité 5 cœurs (CPU0-4) - Nginx

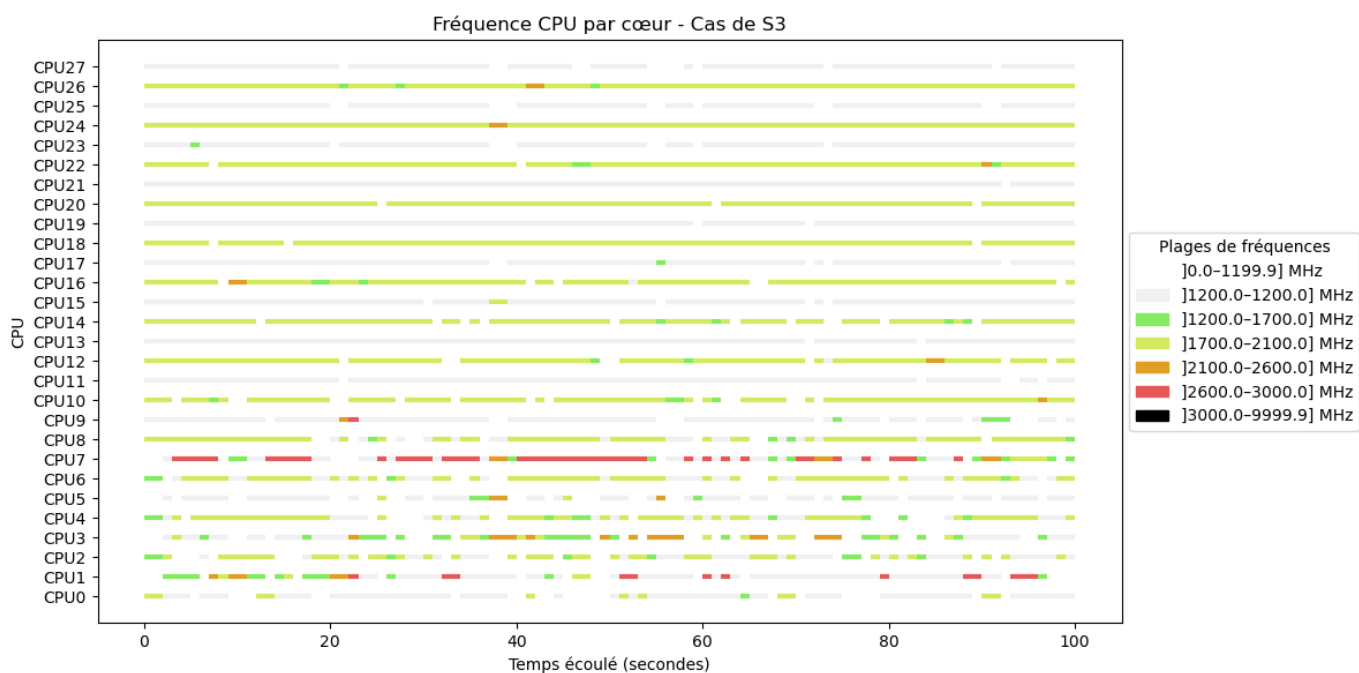


FIGURE 3.8 – Fréquence des cœurs du processeur avec pour affinité 10 cœurs (CPU0-9) - Nginx

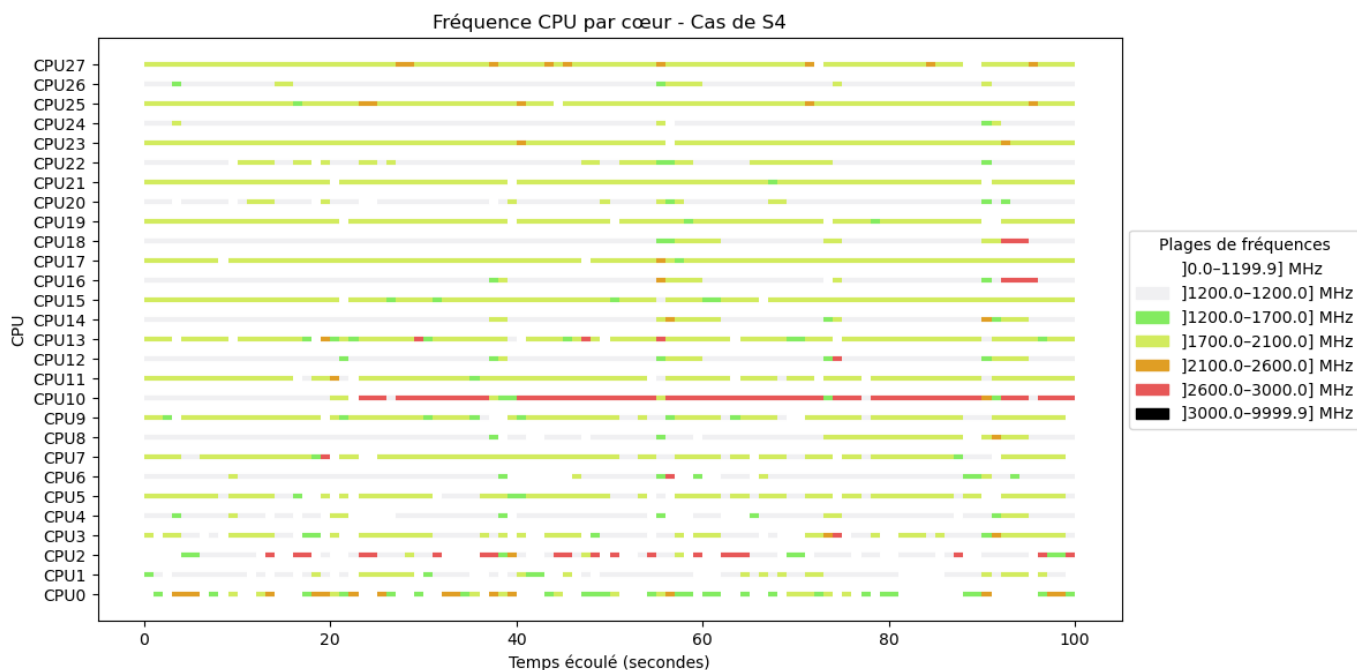


FIGURE 3.9 – Fréquence des cœurs du processeur avec pour affinité 1 cœur différent toutes les secondes - Nginx

Observation

Les résultats obtenus sur les figures 3.5, 3.6, 3.7, 3.8, 3.9, montrent une faible démarcation des fréquences des cœurs sur lesquels sont définies les affinités CPU. Ce qui s'explique par le fait que notre serveur Nginx ne génère pas une activité assez intense pour se différencier. De plus, d'autres programmes à l'instar des programmes Système et des outils de profilage sollicitent également les ressources CPU causant un bruit expérimental masquant les effets spécifiques liés à l'affinité CPU.

— Énergie

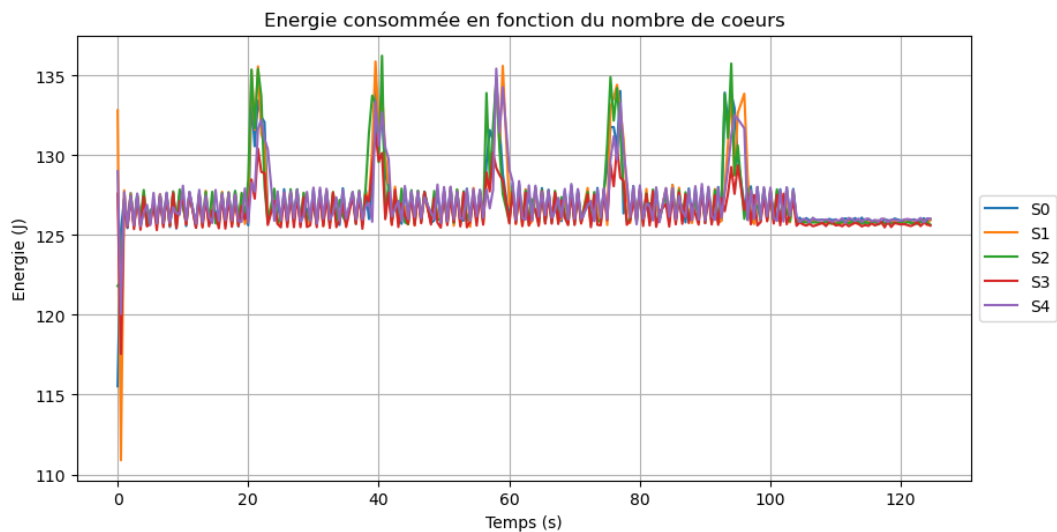


FIGURE 3.10 – Energie consommée - Nginx

Avec pour résultats statistiques :

TABLE 3.9 – Tableau comparatif des résultats statistiques de l’énergie consommée

Affinité CPU	S0	S1	S2	S3	S4
Moyenne	127,101833	127,091208	127,11154	126,5712916	127,1152
Médiane	126,1450	126,0300	125,9850	125,8700	126,1950
99th percentile	134,0010	135,4981	135,4005	130,3264	133,3571

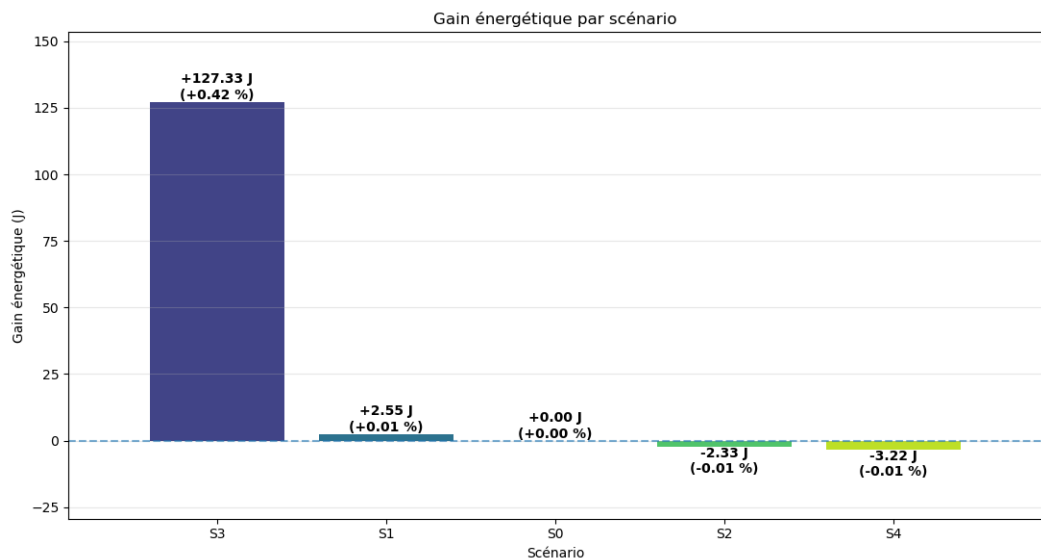


FIGURE 3.11 – Gain énergétique en fonction des scénarios - Nginx

Observation

La courbe S3 est la meilleure énergétiquement, vu qu'elle présente la plus faible moyenne, la plus faible médiane et son 99ème percentile est très inférieur.

3.2.1.3 Indicateurs Système

TABLE 3.10 – Indicateurs Système en fonction de l'affinité CPU - Nginx

Indicateurs Système	S1	S2	S3	S0	S4
context-switches	59.050	51.728	50.917	54.000	54.698
cpu-migrations	1	191	25	100	112
CPUs utilized	0,046	0,048	0,050	0,041	0,042

Observation

Les indicateurs Système montrent une utilisation CPU inférieure à 5% dans tous les scénarios, confirmant le faible impact de notre serveur sur les cœurs. De plus, certains scénarios induisent un nombre bas de migrations, notamment S1 et S3.

3.2.2 Résultats pour le serveur web Node.js

Les expériences menées avec Node.js reposent sur une application réalisant un calcul intensif (Voir Annexe E), avec l'envoi d'une série de soixante requêtes. Contrairement aux résultats observés avec Nginx, les diagrammes de latence et de débit mettent en évidence une sensibilité marquée de Node.js à l'utilisation intensive du CPU.

3.2.2.1 Performances applicatives**— Latence**

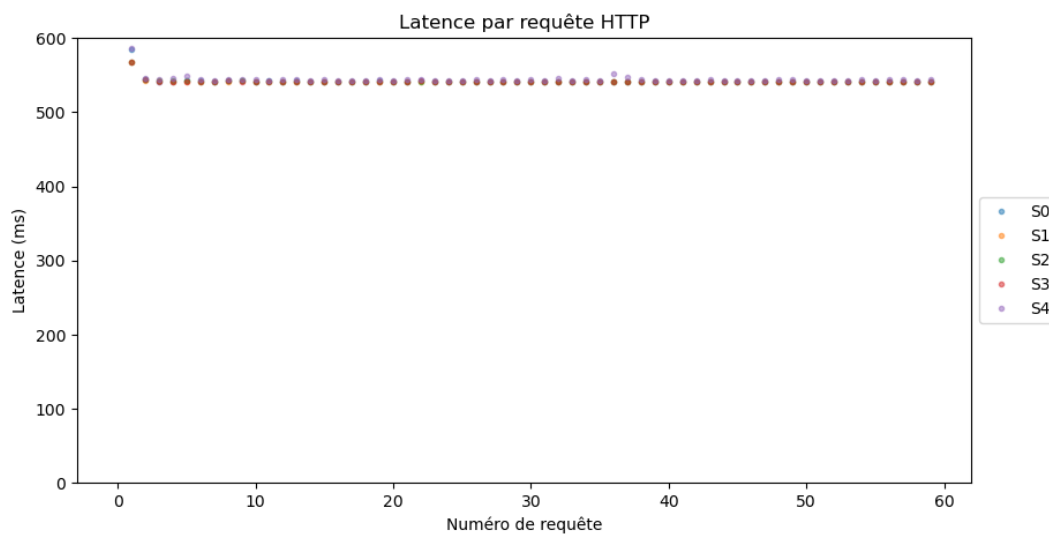


FIGURE 3.12 – Latence des requêtes par affinité CPU - Node.js

Avec pour résultats statistiques :

TABLE 3.11 – Tableau comparatif des résultats statistiques des latences

Scénario	S0	S1	S2	S3	S4
Moyenne	551,190	550,583	551,525	550,601	553,851
Médiane	541,307	541,095	541,369	540,822	543,472
99th percentile	790,153	777,373	792,107	782,245	795,672

Nous observons les caractéristiques suivantes :

- * **S1** : Meilleur Percentile 99, médiane correcte.
- * **S3** : Meilleure médiane, percentile 99 légèrement plus élevé.
- * **S0** et **S2** : Pas de défaut majeur, mais dominées par S1 et S3.
- * **S4** : Pire moyenne, pire médiane et pire Percentile 99.

Observation

Le scénario le plus performant est S1, car il minimise la latence de queue, reste équivalent sur la médiane et présente la distribution la plus saine. Le scénario S3 est un excellent second choix.

— Débit

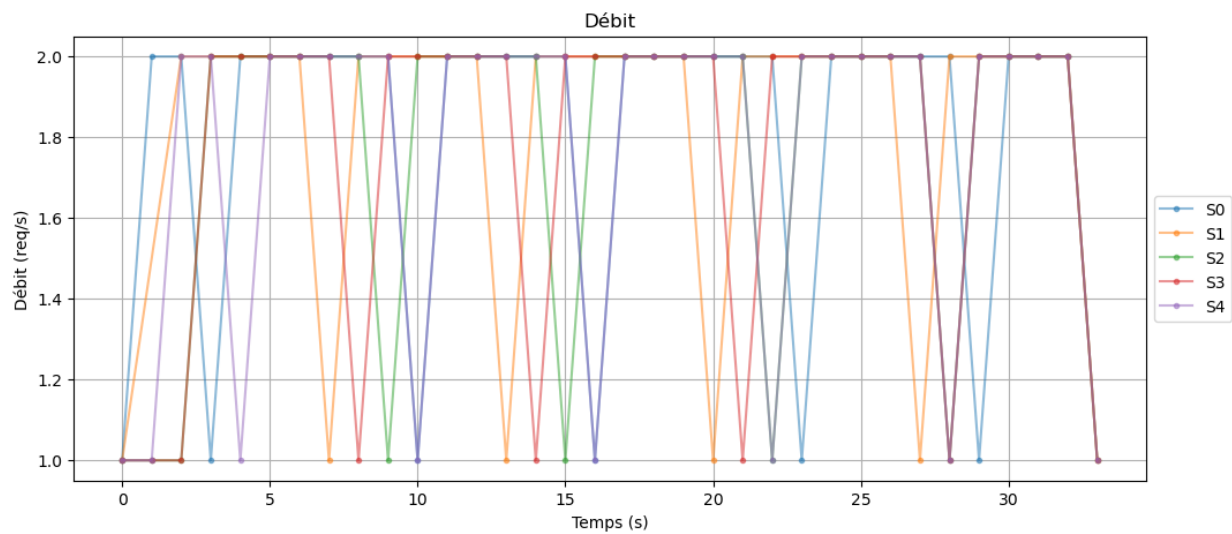


FIGURE 3.13 – Débit des requêtes par affinité CPU - Node.js

Avec pour résultats statistiques :

TABLE 3.12 – Tableau comparatif des résultats statistiques des débits

Scénario	S0	S1	S2	S3	S4
Moyenne	1,818	1,818	1,764	1,764	1,764
Médiane	2	2	2	2	2
99th percentile	2	2	2	2	2

Observation

Les scénarios S0 et S1 ont en moyenne le nombre le plus élevé de requêtes traitées par seconde.

3.2.2.2 Consommation énergétique

— Fréquence

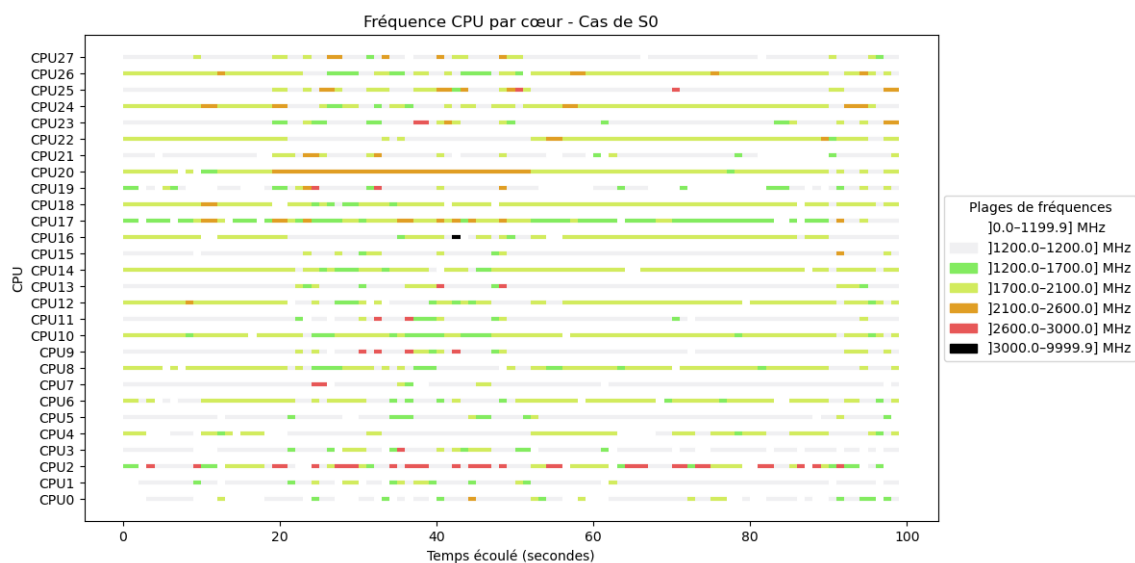


FIGURE 3.14 – Fréquence des cœurs du processeur avec pour affinité tous les cœurs - Node.js

Observation

La figure 3.14 présente des élévations de fréquences plus ou moins éparpillées au sein des cœurs. Ce qui s'explique par la dispersion de la charge sur tous les cœurs due au fait qu'aucune affinité n'a été définie.

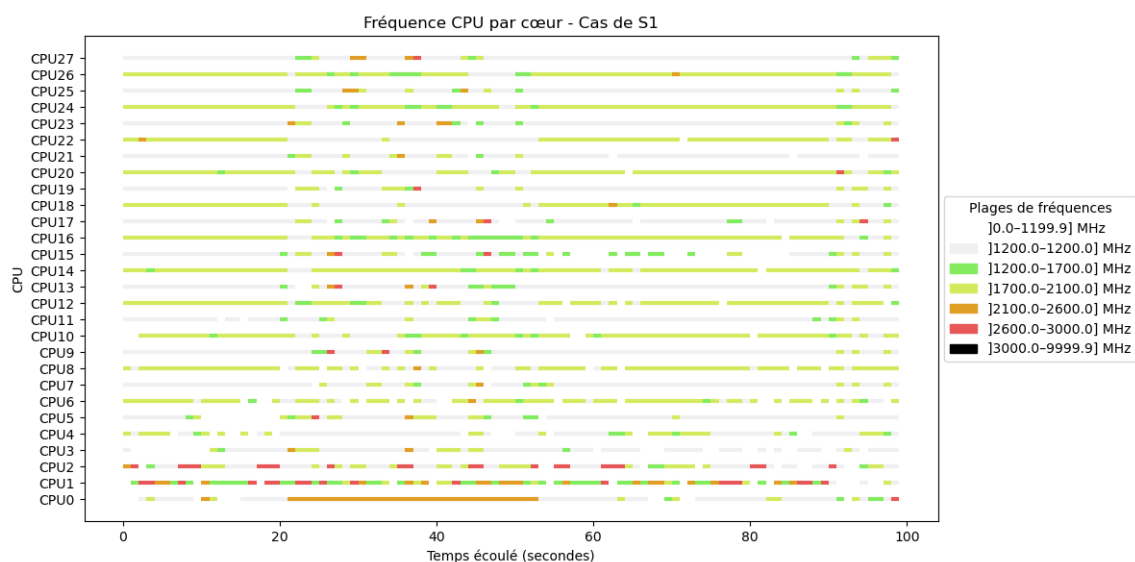


FIGURE 3.15 – Fréquence des cœurs du processeur avec pour affinité 1 cœur (CPU0) - Node.js

Observation

La figure 3.15 présente une élévation de la fréquence du cœur CPU0(Cœur sur lequel a été définie l'affinité). Cette élévation commence après vingt secondes ; heure à laquelle sont envoyées les premières requêtes. Les autres changements de fréquence sont dûs à l'exécution des processus Systèmes et des outils de profilage.

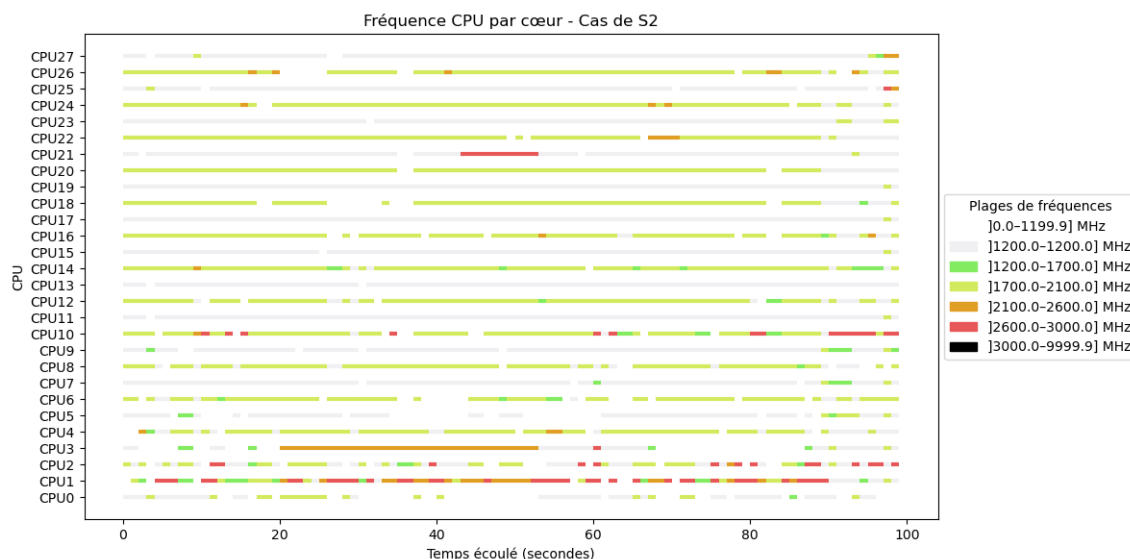


FIGURE 3.16 – Fréquence des cœurs du processeur avec pour affinité 5 cœurs (CPU0-4) - Node.js

Observation

La figure 3.16 présente une élévation de la fréquence du cœur CPU0 et CPU3(Faisant partie des cœurs sur lesquels a été définie l'affinité). Cette élévation commence après vingt secondes ; heure à laquelle sont envoyées les premières requêtes. Les autres changements de fréquence sont dûs à l'exécution des processus Systèmes et des outils de profilage.

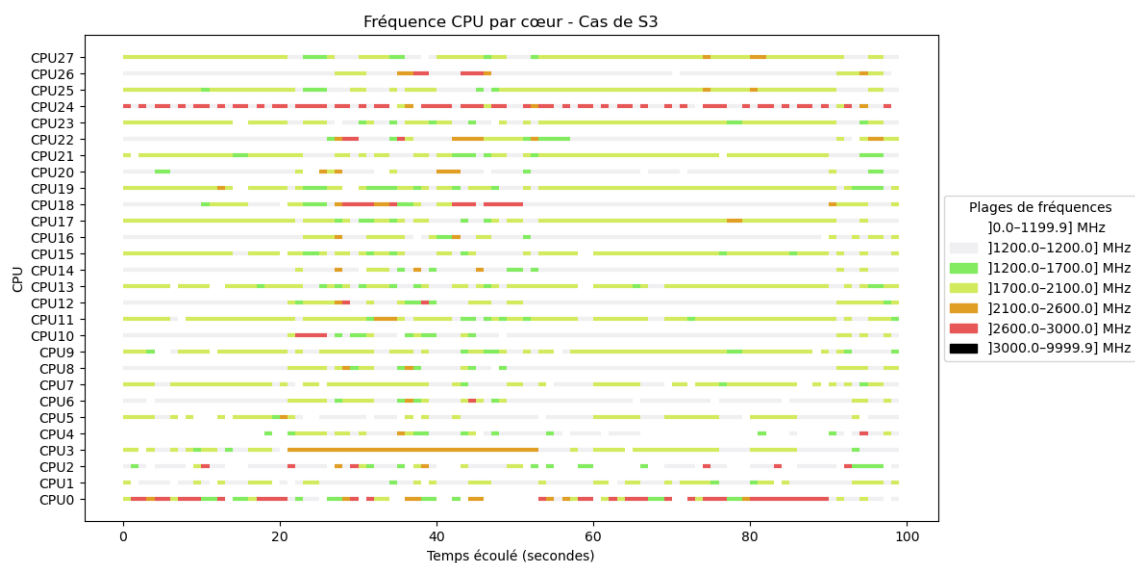


FIGURE 3.17 – Fréquence des cœurs du processeur avec pour affinité 10 cœurs (CPU0-9) - Node.js

Observation

La figure 3.17 présente une élévation de la fréquence du cœur CPU3. Cette élévation commence après vingt secondes ; heure à laquelle sont envoyées les premières requêtes. Les autres changements de fréquence sont dûs à l'exécution des processus Systèmes et des outils de profilage.

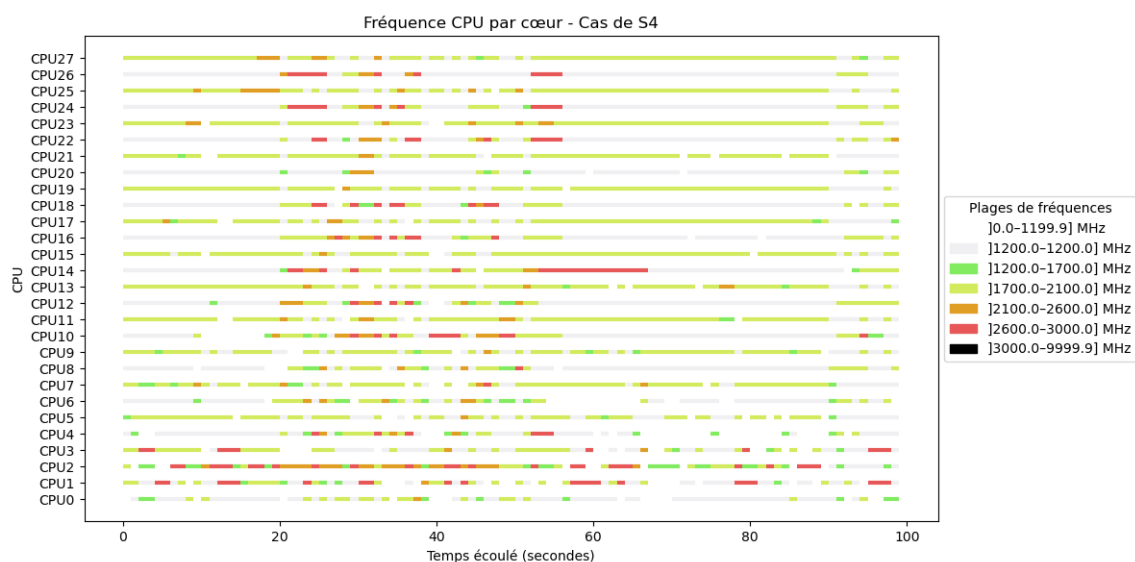


FIGURE 3.18 – Fréquence des cœurs du processeur avec pour affinité 1 cœur différent toutes les secondes - Node.js

Observation

La figure 3.18 présente des élévations successives de la fréquence des cœurs allant de CPU10 à CPU27 puis de CPU0 à CPU13. Cette élévation commence après vingt secondes ; heure à laquelle sont envoyées les premières requêtes. Les autres changements de fréquence sont dus à l'exécution des processus Systèmes et des outils de profilage.

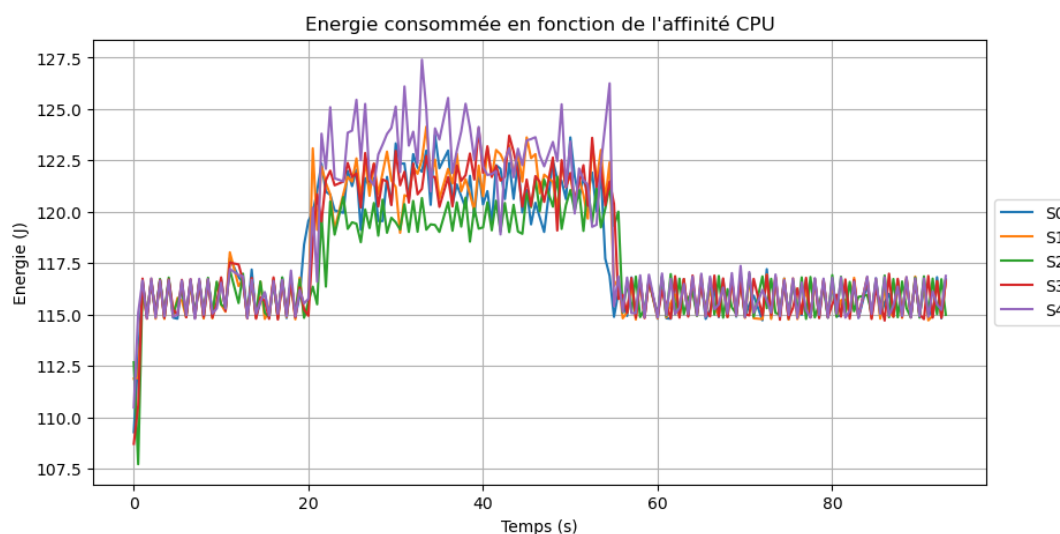
— Energie

FIGURE 3.19 – Energie consommée par le serveur Node.js

Avec pour résultats statistiques :

TABLE 3.13 – Tableau comparatif des résultats statistiques de l'énergie consommée

Affinité CPU	S0	S1	S2	S3	S4
Moyenne	117,693	117,865	117,216	117,841	118,378
Médiane	116,680	116,680	116,725	116,725	116,760
99th percentile	123,391	123,335	121,574	123,623	126,131

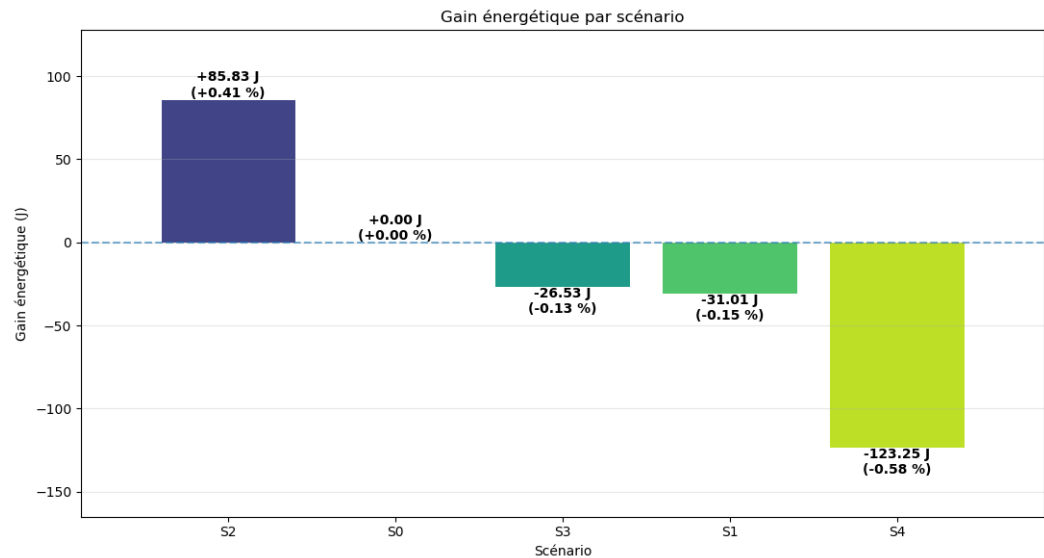


FIGURE 3.20 – Gain énergétique en fonction des scénarios - Node.js

Observation

Le scénario S2 est le plus performant, car il minimise les pics de consommation et consomme moins pour un travail équivalent. S4 est à éviter, car il concentre les pires caractéristiques énergétiques.

3.2.2.3 Indicateurs Système

TABLE 3.14 – Indicateurs Système en fonction de l’affinité CPU - NodeJS

Indicateurs Système	S0	S1	S2	S3	S4
context-switches	1.357	1.559	1.396	1.397	1.708
cpu-migrations	13	0	12	26	40
CPUs utilized	0,557	0,547	0,557	0,557	0,559

Observation

Les scénarios d’affinité fixe montre des valeurs plus favorables pour certaines métriques liées aux cycles et aux instructions, suggérant une exécution plus efficace. En revanche, le scénario d’affinité dynamique se caractérise par une augmentation des événements liés aux migrations et aux défauts de cache, ce qui explique la dégradation observée tant sur le plan des performances que sur celui de la consommation énergétique.

3.2.3 Implications pour les déploiements en production

Les résultats précédents permettent d’identifier des configurations d’affinité CPU présentant des intérêts particuliers pour une exploitation en environnement de production. En particulier, le

scénario S3 pour Nginx qui se distingue par un gain plus élevé de la consommation énergétique et S2 pour Node.js qui se distingue par le compromis le plus favorable entre performances applicatives et consommation énergétique. Ces configurations ont été retenues comme scénarios les plus bénéfiques sur nos serveurs.

Ainsi, au-delà des gains techniques, ces optimisations ont un impact économique mesurable. Les Tableaux 3.15 et 3.16 présentent les économies réalisées grâce aux scénarios les plus bénéfiques sur nos serveurs à savoir, S3 pour Nginx et S2 pour Node.js, sur cent mille (100.000) serveurs pendant un an, en fonction de la grille tarifaire de l'entreprise ENEO [26], à différents niveaux de charge.

TABLE 3.15 – Gains financiers (en FCFA) par an pour 100.000 serveurs - Nginx

Nombre de requêtes	10K	50K	100K	200K	400K	500K
Économies (FCFA)	59	297	594	1188	2376	2970

TABLE 3.16 – Gains financiers (en FCFA) par an pour 100.000 serveurs - NodeJS

Nombre de requêtes	10K	50K	100K	200K	400K	500K
Économies (FCFA)	33.366	182.722	365.444	786.500	1.573.000	1 966 250

Ces économies varient approximativement entre 59 et 2970 FCFA pour l'application Nginx, et entre 33 366 et 1 966 250 FCFA pour l'application Node.js par an pour 100 000 serveurs.

Outre les économies d'énergie réalisées sur le processeur, la réduction de la consommation énergétique se traduit par une température plus basses, ce qui pourrait prolonger la durée de vie d'un serveur (en ralentissant le vieillissement) et réduire les coûts de maintenance [27] tout en réduisant les coûts de refroidissement du serveur.

3.3 DISCUSSION

3.3.1 Evaluation des hypothèses

Cette étude s'appuie sur deux hypothèses principales relatives à l'impact de l'affinité CPU sur la consommation énergétique et les performances des applications web.

3.3.1.1 Validation de l'hypothèse 1

La première hypothèse postule qu'une affectation contrôlée des processus à un sous-ensemble de cœurs permettrait de réduire la consommation énergétique globale du système. Les résultats obtenus montrent une validation partielle de cette hypothèse, dépendante du type d'application considérée. les résultats confirment que :



- Dans le cas de l'application Nginx, les mesures énergétiques indiquent une légère réduction de la consommation pour la configuration à dix cœurs, bien que les autres scénarios présentent des valeurs proches de celles observées sans affinité. Ce résultat suggère qu'une limitation modérée du nombre de cœurs actifs peut contribuer à stabiliser la consommation énergétique, sans toutefois produire de gains significatifs dans ce contexte.
- Dans le cas de l'application Node.js à calcul intensif, l'affinité CPU a permis une diminution plus marquée et plus régulière de la consommation énergétique. La concentration de l'exécution sur un ensemble réduit de cœurs limite les migrations et réduit l'activation inutile de ressources matérielles, ce qui se traduit par une meilleure efficacité énergétique.

Ainsi, l'hypothèse 1 est validée principalement pour les applications fortement sollicitées par le processeur, tandis que son impact reste limité pour les charges orientées entrées/sorties.

3.3.1.2 Validation de l'hypothèse 2

La seconde hypothèse stipule qu'il est possible de réduire la consommation énergétique sans dégrader significativement les performances perçues par l'utilisateur. Les résultats expérimentaux mettent également en évidence une validation nuancée de cette hypothèse.

- Pour Nginx, les meilleures performances ont été observées dans la configuration sans affinité CPU, tandis que la configuration à dix cœurs, bien que légèrement plus économe en énergie, s'accompagne d'une dégradation modérée des indicateurs de latence. Ce compromis indique que, dans ce cas, l'optimisation énergétique s'effectue au détriment d'une partie des performances applicatives.
- À l'inverse, les expériences menées avec Node.js montrent qu'une configuration d'affinité adaptée permet d'améliorer simultanément les performances et l'efficacité énergétique. La réduction des migrations et la stabilité de l'exécution favorisent à la fois une diminution du temps de réponse et une baisse de la consommation.

Ces résultats confirment que la réduction énergétique sans perte notable de qualité de service est réalisable, mais essentiellement pour des applications sensibles à la charge CPU, dont le comportement bénéficie directement d'un placement maîtrisé des processus.

3.3.2 Limites méthodologiques

Malgré la rigueur apportée à la conception expérimentale, cette étude présente plusieurs limites qu'il convient de souligner. Tout d'abord, les expériences ont été menées sur une plateforme matérielle et logicielle spécifique. Les résultats obtenus peuvent donc varier en fonction de l'architecture du processeur, de la version du noyau Linux ou des mécanismes de gestion d'énergie implémentés. Ensuite, les applications étudiées ne couvrent pas l'ensemble des charges de travail possibles. L'application Nginx utilisée repose sur une configuration par défaut et une



charge relativement simple, tandis que l'application Node.js est centrée sur un calcul intensif. D'autres types d'applications, combinant calcul et entrées/sorties de manière plus équilibrée, pourraient présenter des comportements différents. Par ailleurs, les scénarios de charge restent limités en termes de durée et de diversité. Des tests à plus long terme ou sous des charges fluctuantes pourraient révéler des effets non observés dans cette étude. Enfin, bien que les outils de mesure utilisés soient reconnus, toute instrumentation induit un surcoût potentiel susceptible d'influencer marginalement les résultats.

Toutefois ces limites n'invalident pas les conclusions tirées, mais invitent à une interprétation prudente et ouvrent des perspectives pour des travaux futurs.

3.4 BILAN

Ce chapitre a permis d'analyser expérimentalement l'impact de l'affinité CPU sur les performances et la consommation énergétique des applications web exécutées sous Linux, en s'appuyant sur des mesures quantitatives rigoureuses et reproductibles. Les résultats mettent en évidence que l'efficacité de ce mécanisme dépend fortement de la nature de la charge applicative. Les expérimentations menées avec Nginx, basé sur une application majoritairement orientée entrées/sorties, montrent que les performances sont optimales en l'absence d'affinité explicite, tandis qu'une limitation modérée du nombre de cœurs peut conduire à une légère réduction de la consommation énergétique. Toutefois, ces gains restent limités et s'accompagnent d'une dégradation de la latence, indiquant que l'ordonnancement par défaut du noyau Linux est déjà bien adapté à ce type de charge. À l'inverse, les résultats obtenus avec Node.js, dans le cadre d'une application à calcul intensif, confirment l'intérêt d'une affectation contrôlée des processus. Une configuration d'affinité appropriée permet simultanément d'améliorer les performances et de réduire la consommation énergétique, en limitant les migrations entre cœurs et en favorisant la stabilité de l'exécution.

Dans l'ensemble, ce travail montre que l'affinité CPU constitue un levier d'optimisation pertinent, mais conditionnel, dont l'efficacité dépend du profil applicatif et du type de charge traité. Elle ne peut être considérée comme une solution universelle, mais comme un mécanisme complémentaire aux politiques d'ordonnancement existantes. Nous soulignons ainsi l'importance d'une approche différenciée, tenant compte des caractéristiques des applications, pour concevoir des stratégies d'allocation des ressources à la fois performantes et économes en énergie dans les environnements serveurs modernes.

CONCLUSION GÉNÉRALE ET PERSPECTIVES

La croissance continue des services web et des infrastructures cloud s'accompagne d'une augmentation significative de la consommation énergétique, contribuant de manière notable aux émissions de gaz à effet de serre. Dans ce contexte, l'optimisation de l'utilisation des ressources informatiques constitue un enjeu majeur, tant sur le plan économique qu'environnemental. Ce travail s'est inscrit dans cette problématique en étudiant l'impact de la configuration de l'affinité CPU sur les performances et la consommation énergétique des applications web exécutées sous un système d'exploitation Linux.

L'approche adoptée repose sur une méthodologie expérimentale et comparative, visant à analyser de manière rigoureuse les interactions entre le matériel, le système d'exploitation et les applications. Les expériences ont été menées dans un environnement contrôlé, en faisant varier plusieurs scénarios d'affinité CPU et en mesurant des indicateurs de performance, tels que la latence et le débit des requêtes, ainsi que des métriques énergétiques et bas niveau issues d'outils de profilage. Deux serveurs d'applications web représentatifs, Nginx et Node.js, ont été retenus afin d'évaluer l'influence de l'architecture logicielle sur l'efficacité de ces configurations.

Les résultats expérimentaux mettent en évidence des comportements contrastés selon la nature des charges de travail. Dans le cas de Nginx, utilisé avec une application web simple et majoritairement orientée entrées/sorties, les meilleures performances ont été observées en l'absence d'affinité explicite, tandis qu'un cas d'affinité CPU réduite a permis une réduction de la consommation énergétique engendrant un gain de 0,42% d'énergie non consommée. Ces résultats indiquent que, bien que l'affinité CPU n'apporte pas d'amélioration en termes de débit et de latence pour ce type d'applications web, elle peut néanmoins contribuer à une amélioration énergétique. À l'inverse, les expériences menées avec Node.js, dans le cadre d'une application à calcul intensif, montrent que l'affinité CPU constitue un levier d'optimisation pertinent. Une affectation stable et restreinte des processus à un ensemble réduit de cœurs permet d'améliorer simultanément les performances et l'efficacité énergétique offrant un gain de 0,41% d'énergie non consommée, tandis que des configurations impliquant des migrations fréquentes entre cœurs se révèlent défavorables sur tous les plans. Ces observations confirment le rôle central de la stabilité d'exécution et de la limitation des migrations dans l'optimisation énergétique des applications intensives en calcul.

Sur le plan scientifique, ce travail met en évidence le fait que la consommation énergétique d'un système ne dépend pas uniquement des caractéristiques matérielles, mais également des

décisions prises au niveau du système d'exploitation. L'affinité CPU apparaît ainsi comme un mécanisme simple, non intrusif et facilement déployable, dont l'efficacité dépend fortement du profil applicatif. Elle ne constitue pas une solution universelle, mais un levier complémentaire aux politiques d'ordonnancement existantes, particulièrement pertinent pour certaines catégories d'applications.

Cependant, cette étude présente certaines limites. Les expérimentations ont été réalisées sur une plateforme matérielle et logicielle spécifique, et sur un nombre restreint d'applications web. Les résultats obtenus ne peuvent donc pas être généralisés à l'ensemble des charges de travail rencontrées en production. De plus, les expériences ont été conduites dans un environnement physique, sans intégrer les mécanismes de virtualisation et d'orchestration largement utilisés dans les infrastructures cloud modernes.

Ces limites ouvrent plusieurs perspectives d'approfondissement :

- Tout d'abord étendre les expérimentations à d'autres applications web, notamment des applications sensibles au cache, afin d'analyser plus finement l'impact de la localité des données et des hiérarchies mémoire sur les performances et la consommation énergétique.
- Puis reproduire les expériences dans des environnements virtualisés, afin d'évaluer l'influence de l'affinité CPU dans des contextes représentatifs des plateformes cloud.
- Enfin, préparer un article scientifique destiné à la conférence **SOSP 2026**[28], afin de valoriser les résultats obtenus et de contribuer à la réduction de la consommation énergétique des infrastructures numériques, dont l'empreinte environnementale ne cesse de croître.

RÉFÉRENCES

- [1] THE SHIFT PROJECT. *Lean ICT : Pour une sobriété numérique*. Consulté en octobre 2025. The Shift Project, 2019. URL : <https://theshiftproject.org/publications/pour-une-sobriete-numerique/>.
- [2] Andrew S. TANENBAUM et Herbert BOS. *Modern Operating Systems*. Pearson, 2015. URL : <https://theshiftproject.org/app/uploads/2025/01/The-Shift-Project-Quels-mondes-virtuels-pour-quel-monde-reel-Rapport-final-2024.pdf>.
- [3] NATIONS UNIES / UNITED NATIONS. *Net Zero Coalition — Zéro émission nette*. [En ligne]. Consulté le 5 octobre 2025. 2025. URL : <https://www.un.org/fr/climatechange/net-zero-coalition>.
- [4] Redha GOUICEM et al. “Fewer Cores, More Hertz: Leveraging High-Frequency Cores in the OS Scheduler for Improved Application Performance”. In : *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, juill. 2020, p. 435-448. ISBN : 978-1-939133-14-4. URL : <https://www.usenix.org/conference/atc20/presentation/gouicern>.
- [5] TENCENT CLOUD. *What are cloud workloads?* [En ligne]. Consulté le 2 août 2025. 2025. URL : <https://www.tencentcloud.com/techpedia/131616>.
- [6] GEONOV. *Architecture client-serveur*. [En ligne]. Consulté le 24 juin 2025. 2025. URL : <https://www.geonov.fr/architecture-client-serveur/>.
- [7] AMAZON WEB SERVICES (AWS). *Qu’est-ce qu’une application Web ?* Consulté le 12 novembre 2025. URL : <https://www.aws.amazon.com/fr/what-is/web-application/>.
- [8] Cyber GEE. *CPU Affinity*. [En ligne]. Consulté le 6 octobre 2025. 2024. URL : <https://medium.com/@CyberGee/cpu-affinity-69f2046c23c4>.
- [9] NVIDIA ENTERPRISE SUPPORT. *What is CPU Affinity?* [En ligne]. Consulté le 6 octobre 2025. 2025. URL : <https://enterprise-support.nvidia.com/s/article/what-is-cpu-affinity-x>.
- [10] Robert M. LOVE. *taskset(1) — set or retrieve a process’s CPU affinity*. [En ligne]. Consulté le 5 septembre 2025. man7.org — Linux manual pages. 2026. URL : <https://www.man7.org/linux/man-pages/man1/taskset.1.html>.

-
- [11] Ahmad ALSHAREF et al. “Cache Memory: An Analysis on Performance Issues”. In : *Proceedings of the 2021 8th International Conference on Computing for Sustainable Global Development (INDIACom)*. IEEE, 2021, p. 184-188. DOI : 10.1109/INDIACom51348.2021.00033.
- [12] Maxime COLMANT, Romain ROUVOY et Lionel SEINTURIER. “Estimation de la consommation des systèmes logiciels sur des architectures multi-cœurs”. In : *Compas’2015*. Communication dans un congrès, HAL id: hal-01171696. Lille, France, 2015. URL : <https://hal.archives-ouvertes.fr/hal-01171696>.
- [13] Vijayalakshmi SARAVANAN et al. “A Study on Factors Influencing Power Consumption in Multithreaded and Multicore CPUs”. In : (2011). URL : https://www.researchgate.net/publication/228576499_A_study_on_factors_influencing_power_consumption_in_multithreaded_and_multicore_CPUs.
- [14] Chung-Hsing HSU et Ulrich KREMER. *Compiler-Directed Dynamic Voltage Scaling for Memory-Bound Applications*. Rapp. tech. DCS-TR-498. Technical Report. New Brunswick / Piscataway, Camden et Newark : Department of Computer Science, Rutgers University, 2002.
- [15] THE LINUX KERNEL ORGANIZATION, INC. *Rules on how to access information in sysfs*. Consulté le 12 novembre 2025. URL : <https://docs.kernel.org/admin-guide/sysfs-rules.html>.
- [16] THE LINUX KERNEL ORGANIZATION, INC. *Schedutil — CPU frequency scaling governor based on scheduler utilization*. [En ligne]. Consulté le 13 décembre 2025. URL : <https://docs.kernel.org/scheduler/schedutil.html>.
- [17] Glenn K. LOCKWOOD. *Managing Process Affinity in Linux*. [En ligne]. Consulté le 6 octobre 2025. 2025. URL : <https://www.glennklockwood.com/hpc-howtos/process-affinity.html>.
- [18] Werner FISCHER. *Processor P-states and C-states*. [En ligne]. Consulté le 25 octobre 2025. Thomas-Krenn Wiki. 2019. URL : https://www.thomas-krenn.com/en/wiki/Processor_P-states_and_C-states.
- [19] Dmitry DUPLYAKIN, Robert RICCI, Jonathan MORRISON et al. “The Design and Operation of CloudLab”. In : *Proceedings of the USENIX Annual Technical Conference (ATC)*. USENIX Association, juill. 2019, p. 1-14. URL : <https://www.flux.utah.edu/paper/duplyakin-atc19>.
- [20] Ali RAZA et al. “Unikernels: The Next Stage of Linux’s Dominance”. In : *Proceedings of the Workshop on Hot Topics in Operating Systems*. HotOS ’19. Bertinoro, Italy : Association for Computing Machinery, 2019, p. 7-13. ISBN : 9781450367271. DOI : 10.1145/3317550.3321445. URL : <https://doi.org/10.1145/3317550.3321445>.
-

-
- [21] NETCRAFT LTD. *Netcraft Blog*. [En ligne]. Consulté le 25 octobre 2025. 2025. URL : <https://www.netcraft.com/resources/blog>.
- [22] DATADOG. *8 Surprising Facts About Real Docker Adoption*. [En ligne]. Consulté le 1 octobre 2025. 2018. URL : <https://www.datadoghq.com/docker-adoption/>.
- [23] NGINX ORG. *NGINX — High Performance Web Server*. [En ligne]. Consulté le 12 octobre 2025. 2025. URL : <https://nginx.org/en/>.
- [24] NODE.JS FOUNDATION. *Introduction to Node.js*. [En ligne]. Consulté le 25 octobre 2025. 2025. URL : <https://nodejs.org/fr/learn/getting-started/introduction-to-nodejs>.
- [25] A. RUDDLE, C. ALLISON et P. LINDSAY. “Analysing the latency of WWW applications”. In : *Proceedings of the 10th International Conference on Computer Communications and Networks (ICCCN 2001)*. Disponible via ResearchGate, consulté le 16 août 2025. IEEE, 2001, p. 116-121. DOI : 10.1109/ICCCN.2001.956228.
- [26] ENEO CAMEROON S.A. *Tarifs d’électricité*. [En ligne]. Consulté le 2 février 2026. Eneo Cameroon S.A. — MyEasyLight. 2012. URL : <https://www.eneocameroon.cm/index.php/fr/clients-particuliers-vos-factures-et-paiement/clients-particuliers-vos-factures-et-paiement-tarifs-deelectricite>.
- [27] Majid JALILI et al. “Cost-Efficient Overclocking in Immersion-Cooled Datacenters”. In : *Proceedings of the 48th International Symposium on Computer Architecture (ISCA 2021)*. ACM/IEEE, 2021, p. 623-636. DOI : 10.1109/ISCA52012.2021.00055.
- [28] ACM SIGOPS. *SOSP 2026: The 32nd Symposium on Operating Systems Principles*. [En ligne]. Consulté le 14 janvier 2026. 2026. URL : <https://sigops.org/s/conferences/sosp/2026/>.

ANNEXES

ANNEXE A : Dépôt GitHub et ressources expérimentales

L'ensemble des scripts, codes sources, fichiers de configuration, outils d'automatisation et protocoles expérimentaux utilisés dans ce travail est disponible dans le dépôt GitHub suivant :

<https://github.com/Brenda-Wai/Consommation-energetique-affinite-cpu/>